



WAYPOINT CONNECT

COMPLETE PROJECT DOCUMENTATION

Complete Documentation

Complete software project documentation

Project WaypointConnect — delivery planning and tracking for Waypoint Group's Fresh, Style and Tech brands

Event Tech-Triathlon 2026 · Team Dark Horse

Source Generated 2026-10-04 · repository commit c1e3eb6 + working tree

Accuracy rule: every statement is taken from the project files. Anything that could not be verified is marked "Not verified from the provided project files". No passwords or secrets are included.

Contents

1. Executive summary	4
1.1 Project at a glance	4
2. Problem statement	5
2.1 Problem statement	5
3. Proposed solution and objectives	6
3.1 Proposed solution	6
3.2 Main objectives	6
4. Target users and user roles	7
4.1 Target users and user roles	7
5. Functional features	8
5.1 Main system features	8
6. Business workflow and rules	10
6.1 Business workflow	10
6.2 Important business rules (verified in source)	11
7. System architecture	13
7.1 System overview	13
7.2 High-level architecture	14
7.3 Frontend architecture	15
7.4 Backend architecture	15
7.5 API architecture	16
8. Technology stack	19
8.1 Technology stack	19
8.2 System requirements	19
9. Database design	20
9.1 Database architecture	20
9.2 Database overview	20
9.3 Enum values	20
9.4 All models (tables)	21
9.5 Relationships	26
9.6 Important constraints	27
9.7 Important indexes	27
9.8 Database business rules	27
10. ER diagram	29
11. Authentication and authorisation	31

11.1 Authentication architecture	31
11.2 Role-based access control	31
12. Order lifecycle	33
12.1 Order lifecycle	33
13. Deployment architecture	34
13.1 Docker architecture	34
13.2 Deployment architecture	34
14. Setup instructions	36
14.1 Prerequisites	36
14.2 Environment configuration	36
14.3 Repository, install, Prisma, database, seed	37
14.4 Running the API and the web app	37
14.5 Tests and checks	37
14.6 Production build	37
14.7 Running with Docker Compose	38
14.8 Deployment	38
14.9 Troubleshooting and common errors	38
15. Testing	40
15.1 Testing strategy	40
15.2 Executed on 2026-10-04 (Windows 11, Node v24.21.0, pnpm 9.12.0)	40
15.3 Automated test inventory (all passed in V-01)	41
15.4 Recorded in the project files (not re-executed here)	42
15.5 Authentication and authorisation testing	43
15.6 Business-rule testing	43
15.7 Gaps	44
16. Judge walkthrough	45
16.1 Before you start	45
16.2 Story in one line	45
16.3 Steps	45
16.4 Business-day and cut-off logic	48
17. Designathon requirements	50
17.1 Basis of the comparison	50
17.2 Deliverables	50
17.3 Booklet rules recorded in the repository	51
17.4 Screens from the Figma frames	51
17.5 Added after the original design	52
18. Design departures	53
18.1 Significant departures from the original design	53

19. AI disclosure	57
19.1 Purpose of AI usage	57
19.2 AI tools used	57
19.3 Dates and contributors	57
19.4 What AI helped with	57
19.5 What the team implemented and verified	57
19.6 Testing performed	58
19.7 Human responsibility	58
19.8 AI limitations observed in the record	58
19.9 Final verification statement	58
19.10 Disclosure log (from docs/ai-disclosure.md)	58
20. Limitations	62
20.1 Limitations and known issues	62
20.2 Known limitations	62
21. Future improvements	64
21.1 Improvements suggested by the project record	64
22. Conclusion	65
22.1 Conclusion	65

1. Executive summary

1.1 Project at a glance

Name	Waypoint Connect (repository <code>DarkHorse_WaypointConnect</code> , workspace package <code>waypoint-connect</code> v0.1.0)
Event / team	Tech-Triathlon 2026, Team Dark Horse (<code>README.md</code> , root <code>package.json</code>); <code>docs/handoff.md</code> calls it the Hackathon part of Rootcode Tech-Triathlon 2026
Purpose	Delivery planning and tracking for Waypoint Group's three brands (Fresh, Style, Tech): one system for the dispatcher, the loader, the driver (works offline) and the store manager. An administrator role was added later.
Scale in the data	120 outlets (<code>data/outlets.csv</code>), 60 vehicles (<code>data/vehicles.csv</code>), 2 depots (Peliyagoda, Kandy), 188 sign-in accounts (README)
Live demo	https://3-106-77-179.sslip.io (from <code>README.md</code> ; it answered HTTP 200 when the documentation was generated on 2026-10-04). Driver phone app page: <code>/get-app</code>
Repository	https://github.com/Cognix-0/DarkHorse_WaypointConnect (from <code>docs/handoff.md</code> ; whether it is public <i>Not verified from the provided project files.</i>)

2. Problem statement

2.1 Problem statement

Waypoint Group delivers goods for three brands from two depots. Every day each order has to be assigned to a vehicle and a trip without breaking operating rules that the repository records as coming from the Challenge Booklet (`CLAUDE.md` , `packages/shared/src/engine/rules.ts`):

- weight and volume limits per vehicle, chilled goods only on refrigerated (reefer) vehicles, shops that can only be reached by van, and each vehicle serving only its home depot;
- at most two trips per vehicle per day, one brand and one district per trip, and orders are never split;
- time limits: the Fresh window 03:30–08:00 (270 minutes) and a 480-minute day for Style and Tech, plus a weekly fuel quota per vehicle;
- an order cutoff at 16:00 Sri Lanka time.

On a peak day the chilled demand exceeds the refrigerated capacity: the README describes the official peak-day scenario as about 32.8 t of chilled demand against 3 refrigerated trucks and 1 refrigerated van, so some orders cannot legally be served. Those deferrals need a reason, and shops that were skipped before must not be silently skipped again. Four groups of people must see the same facts at the same time, and drivers must be able to keep working when the phone has no signal.

Source note

The original Challenge Booklet and Designathon proposal were **not** among the provided files. This problem statement is reconstructed from `README.md` , `CLAUDE.md` , `docs/handoff.md` and the code. Statements about the booklet are marked as "recorded in the repository".

3. Proposed solution and objectives

3.1 Proposed solution

Waypoint Connect is one web application (a React single-page app that is also an installable PWA) with a Fastify API and a PostgreSQL database. A planning engine in `packages/shared/src/engine` contains every planning rule and is used by both the API and the browser, so the dispatcher, the loader, the driver and the store manager always see the result of the same rules. The system follows the order through four steps that are also shown on the sign-in page: **Order** → **Plan** → **Load** → **Deliver**, followed by the store's receipt confirmation.

3.2 Main objectives

The README and code do not contain a formal objectives list. The objectives below are derived from what the system implements:

1. Let each store order from a goods catalogue (counted in packs) until the 16:00 cutoff, and follow the order to delivery.
2. Let the dispatcher see every order in priority order, auto-allocate a plan, change it by drag and drop with every drop checked by the engine, and publish it with a stated reason for every deferral.
3. Let loaders load by goods line, report problems, and seal and release a trip, with plan changes shown as a to-do list.
4. Let drivers deliver with proof of delivery, including with no signal, without losing or double-sending any event.
5. Let store managers confirm what arrived line by line and answer a deferral.
6. Let an administrator manage accounts, put one driver on each vehicle, check system health and run the demo.

4. Target users and user roles

4.1 Target users and user roles

Role	Accounts (README)	Works on	Web area
Administrator	<code>admin@waypoint.lk</code> (1; more can be created)	Accounts, drivers on vehicles, system health, demo mode	<code>/admin</code>
Dispatcher	<code>dispatcher@waypoint.lk</code> (1)	Both depots: switches between Peliyagoda and Kandy in the console; plans, publishes, tracks	<code>/dispatcher</code>
Loader	<code>loader1.peliyagoda@waypoint.lk</code> ... <code>loader3.kandy@waypoint.lk</code> (3 per depot)	The loader's own depot: loads and seals trips	<code>/loader</code>
Driver	<code>veh001@waypoint.lk</code> ... <code>veh060@waypoint.lk</code> (1 per vehicle)	Signs in as the vehicle; shows the vehicle's assigned driver; delivers (offline capable)	<code>/driver</code>
Store manager	<code>out001@waypoint.lk</code> ... <code>out120@waypoint.lk</code> (1 per store)	Own store only: orders, tracking, receipts	<code>/store</code>

5. Functional features

5.1 Main system features

Dispatcher console (screens D1–D6)

Screen	What it does (README)
D1 Overview	Today's numbers, demand by brand, both depots, what needs attention
D2 Order Queue	Locked orders in priority order with filters and CSV export; orders placed after the 16:00 cutoff greyed out; Auto-allocate & open board
D3 Planning Board	Drag stops between vehicles and trips; every drop is checked by the engine and a refused drop shows Drop blocked with the exact rule; weight, volume and time meters; weekly fuel per vehicle; ETAs per stop
D4 Deferrals	Overload banner, three recovery options re-planned by the engine, the deferral table, explicit confirmation for a shop skipped twice, Confirm & publish
D5 Live Tracking	Trip progress, ETA against each shop's window (on time / at risk / late / offline), alerts with actions, View POD
D6 Capacity Forecast	Next 3 weeks: chilled demand against reefer space and projected deferrals from the same engine

Loader dock tablet (L1–L5)

Choose vehicle (cards with bay, departure and progress) → Load goods (lines in packs grouped by stop, last stop first, tick each line) → Report a problem (damaged, short, wrong pack..., with action carry over / replace / hold, photo and note) → Plan changes (plan v1 → v2 to-do list, mark moved, acknowledge) → Seal & release (seal number, reefer temperature for reefer vehicles; short packs become an order on the next run).

Driver phone app (M1–M8)

Today's runs → Load check → Route progress (Run / Stops / Sync tabs) → Navigate (opens Google Maps) → Stop arrival → Proof of delivery (delivered / partial / refused, count, camera photo shrunk to about 100 kB, receiver, signature) → Report a problem → Offline sync queue → Back online sheet. Every action is saved on the phone first and synchronised later (section on offline sync in the architecture document).

Store manager (SM1–SM9, phone and desktop)

Today (cutoff countdown, today's deliveries, tomorrow, this week, activity) → Place order from the goods catalogue (dry and chilled ordered separately; copy last order; live weight and volume; editable until 16:00) → Order status and tracking (Received → Scheduled → Loaded → Delivered) → Confirm receipt line by line (OK / Short / Damaged) → Order deferred (accept the new day or cancel) → Orders, Receipts and Help.

Administrator console

System (database and API health, working day and clock, today's orders and plan per depot, *Build today's orders*), **Vehicles & drivers** (one driver per vehicle, move with confirmation, add drivers), **Demo mode** (start, extend, reset, end) and **Accounts** (search, switch on/off, reset password shown once, create dispatcher / loader / administrator accounts).

Cross-cutting features

- **Real-time refresh:** every screen refreshes without a reload through a Server-Sent Events signal (`GET /api/changes`); the signal carries no business data.
- **Offline-first driver app:** service worker (`apps/web/public/sw.js`), IndexedDB outbox (`src/offline/idb.ts`) and an idempotent `POST /api/sync` .
- **Plan versions:** every publish stores a snapshot; loaders see the difference and acknowledge it; sealed or departed trips are locked against changes.
- **Fingerprint / Face ID app lock** for the driver app (WebAuthn, local check only; the password and 12-hour token remain the real login).
- **Get the app page** (`/get-app`): Android APK download or Add to Home Screen on iPhone.
- **Sri Lanka time everywhere** (Asia/Colombo) regardless of the device time zone.

6. Business workflow and rules

6.1 Business workflow

- Order.** A store manager places a dry (and, for Fresh stores, a chilled) order from the goods catalogue. Before 16:00 it is for the next operating day; after 16:00 it goes to the following run.
- Lock.** Orders placed before the cutoff are the run's locked orders; later ones are shown greyed out in the queue.
- Plan.** The dispatcher runs Auto-allocate, adjusts the board by drag and drop (each drop checked by the engine), reviews deferrals and presses Confirm & publish. Every affected store is told; each deferred order becomes a carry-over order on the next run.
- Load.** Loaders claim a vehicle trip, tick each goods line (or report a problem), and seal and release the trip.
- Deliver.** The driver confirms the load, departs, and records arrival, proof of delivery or a problem at each stop, with or without signal.
- Confirm.** The store manager confirms receipt line by line; any short or damaged line raises a Store issue alert for the dispatcher.

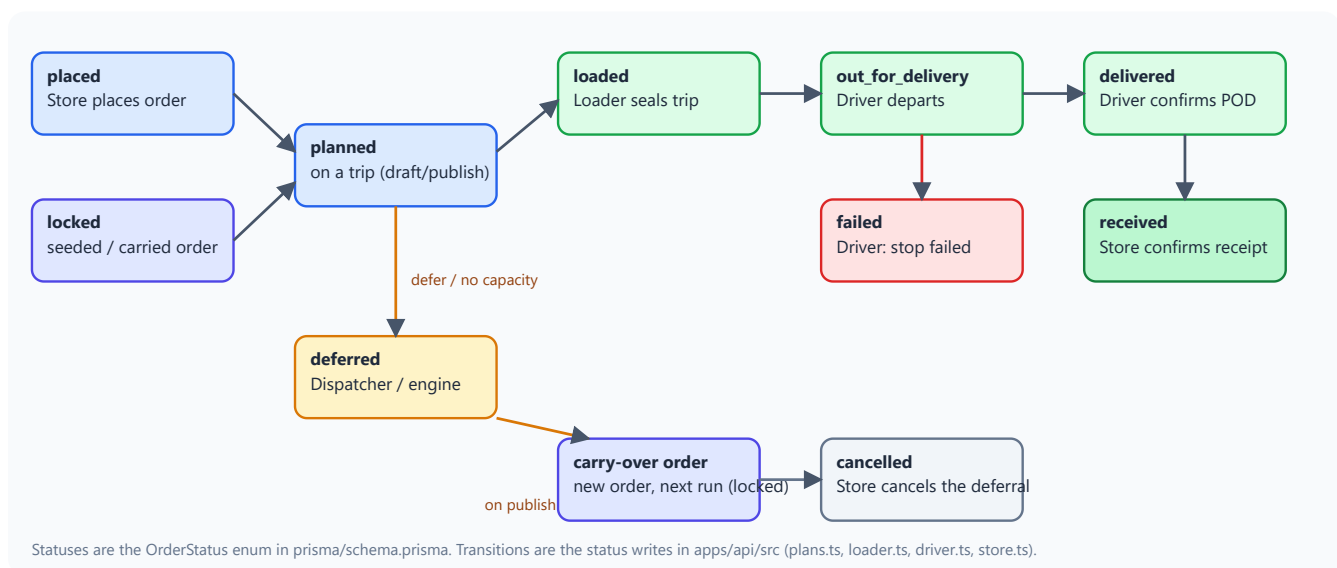
Source: apps/api/src/reference.ts (workingDay, ORDER_CUTOFF, nextOperatingDay) and apps/api/src/routes/store.ts (ctxOf)

"Next operating day" skips non-operating days in data/calendar.csv.



At midnight the calendar day rolls over and the day is built automatically on the first request (builtDays in server.ts).

The 16:00 cut-off and the working run of each role.



Statuses are the OrderStatus enum in prisma/schema.prisma. Transitions are the status writes in apps/api/src (plans.ts, loader.ts, driver.ts, store.ts).

Order status lifecycle (OrderStatus enum and the status writes in the API).

6.2 Important business rules (verified in source)

Rule	Behaviour	Where
Time zone	All business logic and every screen use Asia/Colombo (UTC+5:30). The database stores UTC; the web screens format times with <code>colomboTime</code> / <code>colomboHour</code> .	<code>apps/api/src/reference.ts</code> , <code>apps/web/src/ui/format.ts</code> , <code>docker-compose.yml</code>
Order cutoff	The cutoff is 16:00 on the day before delivery (10:30 UTC).	<code>cutoffFor</code> , <code>ORDER_CUTOFF</code> in <code>reference.ts</code>
Current run (dispatcher, loader, driver)	Today's date before 16:00; from 16:00 the next operating day , whose orders are now locked, so the dispatcher plans and publishes it in the evening.	<code>workingDay()</code> in <code>reference.ts</code>
Store manager day	The store works on the calendar day. Before 16:00 an order is for the next operating day; from 16:00 it goes on the run after that. It can be changed until the cutoff.	<code>ctxOf()</code> in <code>routes/store.ts</code>
Next operating day	Skips days that are not operating days in <code>data/calendar.csv</code> (Sundays and holidays).	<code>nextOperatingDay()</code>
Next-run queue	A store order placed after the cutoff appears at once on the dispatcher's Order Queue and Overview under "next run" (greyed out) until it locks, then in the queue to plan.	README; <code>loadDay()</code> in <code>day.ts</code> (<code>afterCutoff</code>)
Daily order build	Each day is filled with the 85 Peliyagoda orders and workshop vehicles of the official peak-day scenario (S1), built automatically on the first request of a new day and at the run switch. It never adds duplicates.	<code>prisma/demo-day.ts</code> , <code>server.ts</code> (<code>ensureToday</code>)
DEMO_DATE	If set to YYYY-MM-DD, every screen runs on that fixed day instead of today (also <code>DEMO_CLOCK</code> , <code>DEMO_LOADER_CLOCK</code> , <code>DEMO_STORE_CLOCK</code> pin a role's clock). Empty = real date and time.	<code>demoDate()</code> , <code>reference.ts</code> , <code>.env.example</code>
Demo mode (admin)	Runs the whole system on the peak day Wed 25 Mar 2026 for 0.25–12 hours; store at 14:18 on the day before, loader 02:40, dispatcher 06:42. Ends by itself or with End demo now ; Reset demo data returns to the starting point.	<code>apps/api/src/demo.ts</code> , <code>StartDemoRequest</code> in <code>contract.ts</code>
Priority order	1 skipped on the last run (or Fresh shops unserved 2+ days); 2 Fresh chilled; 3 Fresh ambient; 4 Tech; 5 Style. Tie-breaks: van-only, then mall slot, earliest window close, heaviest first.	<code>engine/priority.ts</code>
Vehicle and trip rules	Chilled needs a reefer; van-only outlets need a van; a vehicle serves only its home depot; workshop vehicles are refused; max 2 trips per vehicle per day; one brand and one district per trip; weight and volume caps; weekly fuel quota; orders are never split (one stop per order).	<code>engine/rules.ts</code> (codes such as <code>NEEDS_REEFER</code> , <code>MAX_TRIPS</code>)
Time limits	Trip minutes = depot-to-district free-flow + inter-stop free-flow × (stops – 1) + service allowance per stop. Fresh trips must fit 270 minutes (03:30–08:00); Style and Tech 480 minutes.	<code>engine/rules.ts</code> , <code>engine/schedule.ts</code> ; tests
Deferrals	Every deferral has a reason and a type (<code>unavoidable</code> or <code>choice</code>). A shop skipped twice needs an explicit dispatcher confirmation before publishing. Publishing carries each deferred order to the next run and tells the store.	<code>schema.prisma</code> (<code>Deferral</code>), <code>plans.ts</code>

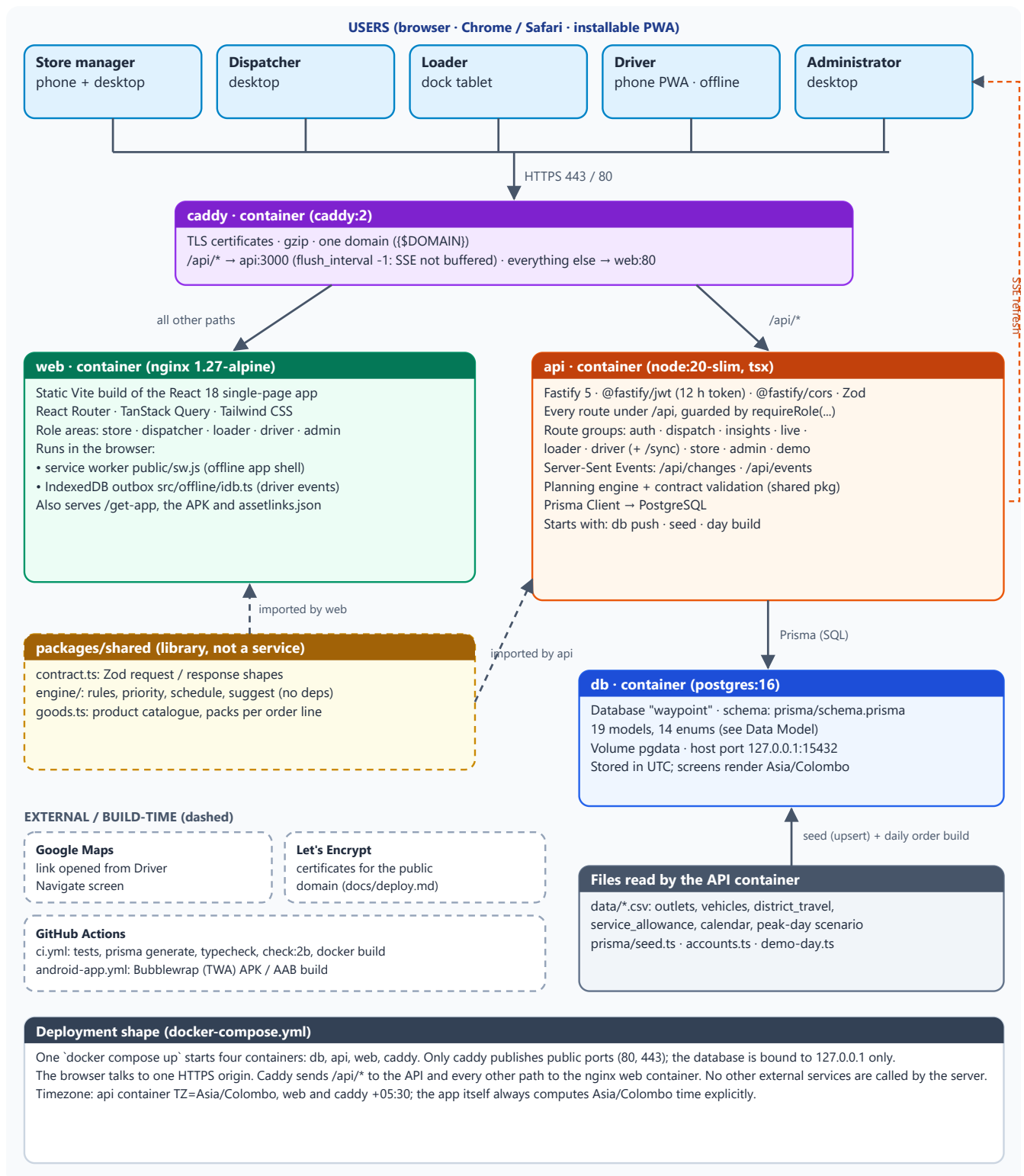
Rule	Behaviour	Where
Locked trips	A sealed or departed trip is locked: the board, defer and auto-allocate refuse to change it.	<code>routes/dispatch.ts</code> (<code>closedTrips</code>), CLAUDE.md
Seal rules	A trip can only be sealed when every line is ticked or reported; reefer vehicles need a logged temperature. Short packs become a new order on the next run.	<code>routes/loader.ts</code> (seal)
Goods in packs	Packs on an order's lines always add up to its units (tested).	<code>packages/shared/src/goods.ts</code>
Admin driver rule	One driver per vehicle; a driver cannot be on two vehicles (database unique index on <code>Vehicle.driverId</code>); moving a driver needs confirmation.	schema, <code>routes/admin.ts</code>
Account switch-off	A switched-off account cannot sign in and an open session stops at its next request.	<code>auth.ts</code> (<code>requireRole</code>)

7. System architecture

7.1 System overview

Waypoint Connect is a pnpm monorepo with three workspaces: `apps/web` (React + Vite), `apps/api` (Fastify) and `packages/shared` (the API contract and the planning engine). At runtime four containers run behind one HTTPS domain. Users reach everything through the Caddy reverse proxy: requests to `/api/*` go to the API, everything else to the nginx container that serves the built web app.

7.2 High-level architecture



Runtime architecture: users, Caddy, web and API containers, PostgreSQL and the files read at start-up. Dashed items are libraries or external/build-time components.

Component	Technology	Responsibility
Users / browsers	Chrome, Safari; installable PWA; Android TWA	Five role areas of one single-page app

Component	Technology	Responsibility
caddy	caddy:2	HTTPS certificates, gzip, routing of <code>/api/*</code> to the API (no buffering for SSE) and everything else to the web container
web	nginx 1.27-alpine serving the Vite build	Static app, cache rules (no-cache for <code>/sw.js</code> and <code>index.html</code>), <code>/get-app</code> , APK and <code>assetlinks.json</code> , <code>/version.json</code>
api	Node 20 + Fastify 5 (run with tsx)	All business logic and routes under <code>/api</code> ; imports <code>@waypoint/shared</code> ; talks to PostgreSQL through Prisma
db	PostgreSQL 16	All persistent state; schema from <code>prisma/schema.prisma</code>
packages/shared	TypeScript, Zod	<code>contract.ts</code> (every request/response shape) and <code>engine/</code> (all planning rules; no npm dependencies so it runs in the browser, the API and the tests)
data/ and prisma/ files	CSV and TypeScript	Reference data and seed. The API reads the CSVs at start-up for travel times, handling allowances and the calendar; the seed upserts outlets, vehicles, drivers and accounts

7.3 Frontend architecture

- **Single-page app** (`apps/web/src/App.tsx`) with React Router. Routes: `/login`, `/get-app`, and one guarded area per role: `/dispatcher/*`, `/loader/*`, `/driver/*`, `/store/*`, `/admin/*`. A `Guard` component redirects to `/login` without a session and to the user's own area when the role does not match.
- **Role folders**: `src/roles/<role>/` hold the screens of one role; shared UI is in `src/ui/` (Badge, Icon, Meter, Modal, States, Toast, format helpers); offline helpers are in `src/offline/` (`idb.ts`, `image.ts`, `sw-register.ts`).
- **Data access**: TanStack Query hooks per role (`roles/*/api.ts`) call `/api` through `src/api.ts`, which keeps the signed-in session (token and user) in the browser's `localStorage`. The Vite dev server proxies `/api` to `http://localhost:3000`.
- **Real-time refresh**: `src/live.tsx` opens the `/api/changes` event stream and refetches when the server signals a change; it catches up after a dropped connection or when the tab returns. A build id with `/version.json` shows a "New version available · Update" bar after a deploy.
- **Offline driver app**: the service worker caches the app shell; the driver's route lives in IndexedDB; every action is an event with a UUID that is applied to the phone's copy and queued in an outbox, then sent in order to `POST /api/sync`. Driver screens never call the API directly (CLAUDE.md rule).
- **Design tokens**: colours, fonts and spacing come from the Tailwind theme tokens copied from Figma (CLAUDE.md rule).
- **Contract**: request and response types are imported from `@waypoint/shared/contract`, so the UI cannot invent fields.

7.4 Backend architecture

The API (`apps/api/src/server.ts`) is a single Fastify process. Plugins: `@fastify/cors` (any origin), `@fastify/jwt` (secret from `JWT_SECRET`). The body limit is 8 MB because driver sync batches carry small photos and signatures. One error shape is used everywhere: `{ "error": "plain sentence", ...details }`.

Request pipeline

1. `onRequest` hook: `ensureToday()` builds today's and the working run's orders if they do not exist yet (once per day and process).
2. Route `preHandler`: `requireRole(...)` verifies the JWT, checks the account is still active and the role matches.
3. Route handler: validates the body with Zod schemas from the contract, uses Prisma and the planning engine.
4. `onResponse` hook: any successful non-GET request (except `/api/auth/*`) calls `signalChange(role)`, which pushes a "something changed" signal to every connected screen.

Source modules

File	Responsibility
<code>auth.ts</code>	<code>requireRole()</code> route guard
<code>day.ts</code>	Loads one delivery day for one depot; maps database rows to engine types and DTOs; order display references (<code>ORD-25xxx</code>)
<code>plans.ts</code>	Writes suggested plans and trips, deferrals, publishing (plan version, snapshot, dock bays, carry-over orders, notifications)
<code>ops.ts</code>	Shared reads for the dock (loader) and the road (driver)
<code>reference.ts</code>	CSV reference data, calendar, cutoff, <code>workingDay()</code> , role clocks
<code>demo.ts</code>	Admin-controlled Demo mode (state stored in the <code>Setting</code> table)
<code>events.ts</code>	In-process event bus for the two SSE streams
<code>routes/*.ts</code>	auth, dispatch, insights, live, loader, driver, store, admin (+ demo), realtime

Planning engine (packages/shared/src/engine)

`rules.ts` (hard rules and `validatePlan`), `priority.ts` (priority policy), `schedule.ts` (stop order by closing time, ETAs, departure times), `suggest.ts` (Suggest plan: places orders in priority order, keeps reefers for chilled goods and vans for van-only outlets, defers what cannot legally fit as `unavoidable` with the reason), `reference.ts` (CSV parsing), `types.ts`. The engine has no npm dependencies.

7.5 API architecture

- REST-style JSON over HTTPS. **Every route lives under `/api`** (registered with prefix `/api`); Caddy forwards `/api/*` to the API container.
- Request and response shapes come only from `packages/shared/src/contract.ts` (Zod). Invalid input answers 400 with a readable message.
- Dispatcher screens work on one day and one depot: `dayOf()` takes the depot from the account; the `?depot=` query applies only to an account without a fixed depot. Loaders use their account depot; drivers use their vehicle's depot; stores are scoped to `outletId`.
- The driver's write path is a single idempotent endpoint, `POST /api/sync`.
- 56 routes in total (listed below, taken from `apps/api/src/routes` and `server.ts`).

Method	Path	Access	Purpose
POST	/api/auth/login	public	Sign in, returns token and session user
GET	/api/health	public	Database ping and server time
GET	/api/auth/me	any signed-in role	Current session user
GET	/api/changes?token=	any signed-in, active	SSE "something changed" signal for every screen
GET	/api/events?token=	any valid token	SSE stream of dispatcher events
GET	/api/orders	dispatcher	Order queue (D2)
GET	/api/board	dispatcher	Planning board data (D3)
POST	/api/plans/suggest	dispatcher	Auto-allocate (Suggest plan)
PUT	/api/plans/trips	dispatcher	Save the board after a drag and drop
POST	/api/plans/check-placement	dispatcher	Check one drop against the rules
POST	/api/orders/:id/defer	dispatcher	Defer an order with a reason
POST	/api/deferrals/:id/confirm	dispatcher	Confirm a second deferral
POST	/api/plans/publish	dispatcher	Publish the plan (raises the version)
GET	/api/overview	dispatcher	Overview (D1)
GET	/api/deferrals	dispatcher	Deferral review (D4)
POST	/api/deferrals/options/:id	dispatcher	Take a recovery option
GET	/api/forecast	dispatcher	Capacity forecast (D6)
GET	/api/live	dispatcher	Live tracking (D5)
GET	/api/live/pod/:stopId	dispatcher	Proof of delivery for a stop
POST	/api/live/alerts/:id/action	dispatcher	Act on an alert
POST	/api/live/simulate	dispatcher	Demo-only simulation (needs DEMO_MODE=1)
GET	/api/loader/vehicles	loader	Vehicles on the published plan (L1)
GET	/api/loader/trips/:id	loader	Goods lines of a trip (L2)
POST	/api/loader/trips/:id/claim	loader	Claim a trip to load
POST	/api/loader/lines/:id/tick	loader	Tick a goods line
POST	/api/loader/trips/:id/problems	loader	Report a problem (L3)
POST	/api/loader/trips/:id/seal	loader	Seal and release (L5)
GET	/api/loader/changes	loader	Plan changes v1 → v2 (L4)
POST	/api/loader/changes/:orderId/moved	loader	Mark an order moved
POST	/api/loader/changes/ack	loader	Acknowledge the new plan version

Method	Path	Access	Purpose
GET	/api/driver/route	driver	The vehicle's route, trips and stops
POST	/api/sync	driver	Idempotent batch of offline events
GET	/api/store/today	store	Today screen (SM1)
GET	/api/store/catalogue	store	Catalogue and last order for ordering (SM2)
POST	/api/store/catalogue/products	store	Add a product to the catalogue
POST	/api/store/orders	store	Place or replace an order
GET	/api/store/orders	store	Order history (SM7)
GET	/api/store/orders/:id	store	Order status and tracking (SM3/SM4)
POST	/api/store/orders/:id/receipt	store	Confirm receipt (SM5)
POST	/api/store/orders/:id/deferral	store	Accept or cancel a deferral (SM6)
GET	/api/store/receipts	store	Receipts (SM8)
GET	/api/store/notifications	store	Notifications
POST	/api/store/notifications/read	store	Mark notifications read
GET	/api/admin/system	admin	System health and today per depot
POST	/api/admin/system/rebuild-today	admin	Build today's orders
GET	/api/admin/accounts	admin	All accounts and counts
POST	/api/admin/accounts	admin	Create a staff account
PATCH	/api/admin/accounts/:id	admin	Switch on/off, rename
POST	/api/admin/accounts/:id/reset-password	admin	Reset a password (shown once)
GET	/api/admin/fleet	admin	Vehicles and drivers
POST	/api/admin/drivers	admin	Add a driver
PUT	/api/admin/vehicles/:id/driver	admin	Assign a driver to a vehicle
GET	/api/admin/demo	admin	Demo mode status and story
POST	/api/admin/demo/start	admin	Start or extend demo mode
POST	/api/admin/demo/reset	admin	Reset demo data
POST	/api/admin/demo/end	admin	End demo mode

8. Technology stack

8.1 Technology stack

Layer	Technology	Version source
Language	TypeScript, strict, everywhere (pnpm workspaces)	root <code>package.json</code> : typescript ^5.6.3
Frontend	React 18, Vite 6, Tailwind CSS 3, React Router 6, TanStack Query 5	<code>apps/web/package.json</code>
Offline / PWA	Service worker <code>public/sw.js</code> , IndexedDB outbox (no dependency), web manifest	<code>apps/web</code>
Backend	Fastify 5, @fastify/jwt 9, @fastify/cors 10, Zod 3, bcryptjs 2, tsx	<code>apps/api/package.json</code>
ORM / database	Prisma 6 and PostgreSQL 16	<code>prisma/schema.prisma</code> , <code>docker-compose.yml</code> (<code>postgres:16</code>)
Shared library	Zod API contract + dependency-free planning engine	<code>packages/shared</code>
Container runtime	Docker Compose: db, api (<code>node:20-slim</code>), web (<code>nginx:1.27-alpine</code>), caddy (<code>caddy:2</code>)	<code>docker-compose.yml</code> , Dockerfiles
Tests	Node built-in test runner (<code>node --import tsx --test</code>)	<code>packages/shared/package.json</code>
CI	GitHub Actions: tests, Prisma generate, typecheck, official allocation checker, Docker build	<code>.github/workflows/ci.yml</code>
Android app	Trusted Web Activity built with Bubblewrap in GitHub Actions (package id <code>lk.darkhorse.waypointconnect</code>)	<code>docs/handoff.md</code> , <code>android-app.yml</code>

8.2 System requirements

Requirement	Value	Source
Node.js	20 or newer	root <code>package.json</code> <code>engines</code> ; Dockerfiles use Node 20. The verification run for this document used Node v24.21.0.
Package manager	pnpm 9.12.0 (enable with <code>corepack enable</code>)	<code>packageManager</code> ; README
Docker	Docker with Compose v2 (<code>docker compose</code>); Docker 29.8.1 was present on the verification machine	README, <code>docs/deploy.md</code>
PostgreSQL	16 (from Docker), or any local PostgreSQL with <code>DATABASE_URL</code> set	README "Develop"
Python 3 + pandas	Only for <code>pnpm check:2b</code> (official checker)	<code>.github/workflows/ci.yml</code>
Browser	Current Chrome or Safari; HTTPS is required for the offline service worker and the camera	<code>docs/deploy.md</code> , <code>Caddyfile</code>
Server (public deployment)	2 vCPU, 2 GB RAM, 20 GB disk, ports 22, 80, 443	<code>docs/deploy.md</code>

9. Database design

9.1 Database architecture

PostgreSQL 16 in the `db` container, accessed only through Prisma Client 6. The schema (`prisma/schema.prisma`) has **19 models** and **14 enums**. Docker start-up runs `prisma/pre-push.sql` (adds `Vehicle.driverId` and its unique index safely on an older database), then `prisma db push --skip-generate`, then the idempotent seed. The database stays on UTC; timestamps are shown in Sri Lanka time by the app. The full description and ER diagram are in the Data Model document.

9.2 Database overview

The system stores all state in one PostgreSQL 16 database named `waypoint`, defined by `prisma/schema.prisma` (Prisma 6, `provider = "postgresql"`, connection from `DATABASE_URL`). The schema contains **19 models** and **14 enums**. Reference data (outlets, vehicles) is loaded from the competition CSVs by `prisma/seed.ts`; orders, plans and everything operational are created by the application. Dates that are calendar days use `@db.Date`; timestamps are stored in UTC.

- **How the schema reaches the database:** the API container runs `prisma db push` at every start (not migrations). An untracked `prisma/migrations/20261004071614_init` folder exists, but it creates only 17 tables (no `Driver` or `Setting`) and its `Role` enum lacks `admin`, so it does not match the current schema.
- **Primary keys:** string ids (`OUT001`, `VEH001`, `DRV001`) for reference data and cuid for generated rows; `DeliveryEvent` uses the client-generated UUID; two tables use composite or natural keys (`VehicleDayStatus`, `Setting`).
- **Not in the schema:** no many-to-many relations are declared; three tables (`PlanVersion`, `PlanAck`, `LoadMove`) refer to a plan by a plain `planId` column without a relation.

9.3 Enum values

Enum	Values	Used by
Role	<code>dispatcher</code> <code>loader</code> <code>driver</code> <code>store</code> <code>admin</code>	User.role, PlanAck.role, Notification.role
Brand	<code>Fresh</code> <code>Style</code> <code>Tech</code>	Outlet.brand, Trip.brand
Temp	<code>ambient</code> <code>chilled</code>	Order.temp
DockType	<code>rear_dock</code> <code>street</code> <code>mall_bay</code>	Outlet.dockType
Parking	<code>normal</code> <code>van_only</code> <code>mall_dock</code>	Outlet.parking
Depot	<code>Peliyagoda</code> <code>Kandy</code>	Outlet.depot, Vehicle.depot, Plan.depot, User.depot
VehicleType	<code>truck</code> <code>van</code>	Vehicle.type
VehicleTemp	<code>reefer</code> <code>ambient</code>	Vehicle.temp

Enum	Values	Used by
OrderStatus	placed locked planned deferred loaded out_for_delivery delivered failed received cancelled	Order.status
PlanStatus	draft published	Plan.status
DeferralReason	no_reefer_capacity vehicle_full time_window fuel_quota no_vehicle other	Deferral.reason
DeferralType	unavoidable choice	Deferral.type
StopStatus	pending arrived delivered failed	TripStop.status
LoadStatus	to_load loading sealed	Trip.loadStatus

9.4 All models (tables)

Optional fields are marked with `?`. "Relationships" lists foreign keys on the model and the inverse sides declared by Prisma.

Model: Outlet table "Outlet"	
Purpose	A store that orders goods (reference data from <code>data/outlets.csv</code>).
Primary key	<code>id</code> (string, e.g. OUT001)
Important fields	<code>brand</code> , <code>district</code> , <code>depot</code> , <code>dockType</code> , <code>parking</code> , <code>mallWindow?</code> , <code>windowOpen</code> / <code>windowClose</code> (HH:MM), <code>lastServedOn?</code> , <code>managerName?</code> , <code>managerPhone?</code> , <code>deliveryNote?</code>
Relationships	One-to-many: <code>orders</code> (Order), <code>users</code> (store-manager User)
Business purpose	Defines where, when and how goods can be delivered; drives van-only, depot and time-window rules and fairness (last served).

Model: Vehicle table "Vehicle"	
Purpose	A truck or van (from <code>data/vehicles.csv</code>).
Primary key	<code>id</code> (e.g. VEH001)
Important fields	<code>type</code> , <code>temp</code> (reefer/ambient), <code>weightCapKg</code> , <code>volumeCapM3</code> , <code>depot</code> , <code>kmPerL</code> , <code>weeklyFuelQuotaL</code> , <code>fuelUsedWeekL</code> , <code>driverName?</code> (copy), <code>driverId?</code> (unique)
Relationships	FK <code>driverId</code> → Driver (one-to-one, optional). One-to-many: <code>dayStatus</code> , <code>trips</code> , <code>users</code>
Business purpose	The capacity the planning engine allocates; the vehicle account is how a driver signs in.

Model: Driver table "Driver"

Purpose	A person who drives; the admin assigns at most one driver per vehicle.
Primary key	<code>id</code> (e.g. DRV001)
Important fields	<code>name</code> , <code>phone?</code> , <code>licenseNo?</code> , <code>createdAt</code>
Relationships	Inverse side of one-to-one with Vehicle (<code>vehicle?</code>)
Business purpose	Lets the administrator manage who drives which vehicle; 60 drivers plus 6 spares are seeded.

Model: VehicleDayStatus table "VehicleDayStatus"

Purpose	Workshop days. No row means the vehicle is available.
Primary key	Composite (<code>vehicleId</code> , <code>date</code>)
Important fields	<code>status</code> (in_workshop), <code>note?</code> , <code>setById?</code>
Relationships	FK <code>vehicleId</code> → Vehicle (many-to-one)
Business purpose	Vehicles in the workshop are refused by the engine (<code>VEHICLE_UNAVAILABLE</code>).

Model: User table "User"

Purpose	A sign-in account of any role.
Primary key	<code>id</code> (cuid)
Important fields	<code>email</code> (unique), <code>name</code> , <code>passwordHash</code> , <code>role</code> , <code>depot?</code> , <code>outletId?</code> , <code>vehicleId?</code> , <code>active</code> , <code>customPassword</code> , <code>lastLoginAt?</code> , <code>createdAt</code>
Relationships	FK <code>outletId</code> → Outlet (optional), FK <code>vehicleId</code> → Vehicle (optional)
Business purpose	Authentication and scoping: store accounts carry an outlet, driver accounts a vehicle, loaders and fixed-depot dispatchers a depot; <code>active=false</code> switches an account off.

Model: Order table "Order"

Purpose	A store order for one delivery date and one temperature class.
Primary key	<code>id</code> (cuid)
Important fields	<code>outletId</code> , <code>deliveryDate</code> , <code>temp</code> , <code>units</code> , <code>weightKg</code> , <code>volumeM3</code> , <code>status</code> , <code>note?</code> , <code>placedAt</code> , <code>sourceRef?</code> (e.g. S1-003, <code>carry:</code> , <code>short:</code>)
Relationships	FK <code>outletId</code> → Outlet. One-to-one: <code>stop</code> (TripStop?), <code>receipt</code> (Receipt?). One-to-many: <code>deferrals</code> , <code>loadChecks</code> , <code>lines</code> . Index (<code>deliveryDate</code> , <code>status</code>)
Business purpose	The unit of planning: never split; placed before/after cutoff by <code>placedAt</code> ; carried-over and short orders reference their origin in <code>sourceRef</code> .

Model: OrderLine table "OrderLine"

Purpose	A goods line of an order, counted in packs.
Primary key	<code>id</code> (cuid)
Important fields	<code>orderId</code> , <code>seq</code> , <code>product</code> , <code>pack</code> , <code>packSize</code> , <code>packs</code> , <code>ordered</code> , <code>weightKg</code> , <code>loadedAt?</code> , <code>shortPacks</code> , <code>problem?</code>
Relationships	FK <code>orderId</code> → Order (many-to-one, cascade delete). Index <code>orderId</code>
Business purpose	What loaders tick, drivers deliver and stores confirm; packs on all lines add up to <code>Order.units</code> (enforced by <code>goods.ts</code> , not by the database).

Model: Plan table "Plan"

Purpose	The plan for one depot on one date.
Primary key	<code>id</code> (cuid)
Important fields	<code>date</code> , <code>depot</code> , <code>status</code> (draft/published), <code>version</code> (0 until first published), <code>publishedAt?</code> , <code>options</code> (Json)
Relationships	One-to-many: <code>trips</code> , <code>deferrals</code> . Unique (<code>date</code> , <code>depot</code>)
Business purpose	Holds the dispatcher's plan; loaders and drivers only work from a plan with <code>version > 0</code> ; a new version makes drivers refetch.

Model: Trip table "Trip"

Purpose	One trip of one vehicle (1 or 2 per day) for one brand and district.
Primary key	<code>id</code> (cuid)
Important fields	<code>planId</code> , <code>vehicleId</code> , <code>tripNo</code> , <code>brand</code> , <code>district</code> , <code>departAt</code> , <code>minutes</code> , <code>loadStatus</code> , <code>bay</code> , <code>loaderId?</code> , <code>loaderName?</code> , <code>loadingStartedAt?</code> , <code>sealedAt?</code> , <code>sealNo?</code> , <code>reeferTempC?</code> , <code>heldAt?</code> , <code>departedAt?</code>
Relationships	FK <code>planId</code> → Plan (cascade), FK <code>vehicleId</code> → Vehicle. One-to-many: <code>stops</code> , <code>loadChecks</code> . Unique (<code>planId</code> , <code>vehicleId</code> , <code>tripNo</code>)
Business purpose	Encodes max 2 trips per vehicle, one brand and one district per trip; <code>sealedAt</code> / <code>departedAt</code> lock the trip.

Model: TripStop table "TripStop"

Purpose	An order placed on a trip, in delivery sequence.
Primary key	<code>id</code> (cuid)
Important fields	<code>tripId</code> , <code>orderId</code> (unique), <code>seq</code> , <code>plannedArrival</code> , <code>status</code> , <code>arrivedAt?</code> , <code>deliveredAt?</code> , <code>deliveredUnits?</code> , <code>receiverName?</code> , <code>failedReason?</code>
Relationships	FK <code>tripId</code> → Trip (cascade), FK <code>orderId</code> → Order (one-to-one). One-to-many: <code>events</code> (DeliveryEvent)
Business purpose	Unique <code>orderId</code> means an order is on at most one stop, so orders are never split.

Model: Deferral table "Deferral"

Purpose	A record that an order was not served on its run, with the reason.
Primary key	<code>id</code> (cuid)
Important fields	<code>orderId</code> , <code>planId?</code> , <code>reason</code> , <code>type</code> , <code>note?</code> , <code>detail?</code> , <code>newDate</code> , <code>byUserId?</code> , <code>confirmedAt?</code> , <code>storeResponse?</code> (accepted cancelled), <code>respondedAt?</code> , <code>createdAt</code>
Relationships	FK <code>orderId</code> → Order, FK <code>planId</code> → Plan (optional). Index <code>orderId</code>
Business purpose	Every deferral has a reason and is <code>unavoidable</code> or <code>choice</code> ; a second deferral needs <code>confirmedAt</code> before publishing; the store answers on SM6.

Model: LoadCheck table "LoadCheck"

Purpose	A loader's problem report on a goods line (L3).
Primary key	<code>id</code> (cuid)
Important fields	<code>tripId</code> , <code>orderId</code> , <code>lineId?</code> , <code>loadedUnits</code> , <code>shortUnits</code> , <code>problem?</code> , <code>action?</code> (carry_over replace hold), <code>note?</code> , <code>photo?</code> , <code>byUserId?</code> , <code>byName?</code> , <code>at</code>
Relationships	FK <code>tripId</code> → Trip (cascade), FK <code>orderId</code> → Order. <code>lineId</code> is a plain column (no relation)
Business purpose	Records what went wrong at the dock; short packs are carried to the next run when the trip is sealed.

Model: DeliveryEvent table "DeliveryEvent"

Purpose	One client event from the driver app (offline-first).
Primary key	<code>eventId</code> (client UUID)
Important fields	<code>stopId</code> , <code>type</code> (stop.arrived stop.delivered stop.failed), <code>occurredAt</code> , <code>receivedAt</code> , <code>deviceId</code> , <code>receiverName?</code> , <code>deliveredUnits?</code> , <code>reason?</code> , <code>note?</code> , <code>photo?</code> , <code>signature?</code> , <code>outcome?</code>
Relationships	FK <code>stopId</code> → TripStop (many-to-one)
Business purpose	The primary key makes <code>POST /api/sync</code> idempotent: a repeated event id is skipped.

Model: PlanVersion table "PlanVersion"

Purpose	What each published version of a plan looked like.
Primary key	<code>id</code> (cuid)
Important fields	<code>planId</code> , <code>version</code> , <code>snapshot</code> (Json), <code>publishedAt</code> , <code>byName?</code> ; unique (<code>planId</code> , <code>version</code>)
Relationships	<code>planId</code> is a plain column: no Prisma relation to Plan
Business purpose	Lets the dock see "v1 → v2" changes.

Model: PlanAck table "PlanAck"

Purpose	Who confirmed they work from which plan version.
Primary key	<code>id</code> (cuid)
Important fields	<code>planId</code> , <code>version</code> , <code>userId</code> , <code>userName</code> , <code>role</code> , <code>at</code> ; unique (<code>planId</code> , <code>version</code> , <code>userId</code>)
Relationships	Plain columns, no relations
Business purpose	The dispatcher sees who acknowledged a plan change and when.

Model: LoadMove table "LoadMove"

Purpose	A loader moved an order's goods to its new vehicle after a plan change (L4 "Mark moved").
Primary key	<code>id</code> (cuid)
Important fields	<code>planId</code> , <code>version</code> , <code>orderId</code> , <code>byName?</code> , <code>movedAt</code> ; unique (<code>planId</code> , <code>version</code> , <code>orderId</code>)
Relationships	Plain columns, no relations
Business purpose	Tracks the physical re-loading after a republish.

Model: Receipt table "Receipt"

Purpose	The store manager's confirmation of what arrived (SM5).
Primary key	<code>id</code> (cuid)
Important fields	<code>orderId</code> (unique), <code>confirmed</code> , <code>issue?</code> , <code>issueUnits?</code> , <code>note?</code> , <code>lines</code> (Json: ok short damaged per line), <code>deliveredAt?</code> , <code>byName?</code> , <code>at</code>
Relationships	FK <code>orderId</code> → Order (one-to-one)
Business purpose	Closes the loop; a short or damaged line raises a "Store issue" alert for the dispatcher.

Model: Notification table "Notification"

Purpose	Messages to stores and alerts for the dispatcher.
Primary key	<code>id</code> (cuid)
Important fields	<code>role?</code> , <code>outletId?</code> , <code>userId?</code> , <code>kind</code> , <code>title?</code> , <code>message</code> , <code>refs</code> (Json), <code>read</code> , <code>done</code> , <code>createdAt</code> . Indexes (<code>outletId</code> , <code>read</code>), (<code>role</code> , <code>createdAt</code>)
Relationships	No relations (plain columns)
Business purpose	Store notifications (plan published, deferrals, deliveries) and dispatcher alerts (Live Tracking).

Model: Setting table "Setting"	
Purpose	Small system settings set by the admin, e.g. the <code>demo</code> key.
Primary key	<code>key</code> (string)
Important fields	<code>value</code> (Json), <code>updatedAt</code>
Relationships	None
Business purpose	Persists Demo mode state (on/off, day, clocks) across API restarts.

9.5 Relationships

One-to-one

Relationship	How it is enforced
Vehicle ↔ Driver (optional on both sides)	<code>Vehicle.driverId</code> is <code>@unique</code> : a driver is never on two vehicles
Order ↔ TripStop (an order has 0 or 1 stop)	<code>TripStop.orderId</code> is <code>@unique</code> : orders are never split
Order ↔ Receipt (0 or 1)	<code>Receipt.orderId</code> is <code>@unique</code>

One-to-many

"One" side	"Many" side (FK)	Notes
Outlet	Order.outletId	
Outlet	User.outletId	optional: store-manager accounts
Vehicle	User.vehicleId	optional: driver (vehicle) accounts
Vehicle	Trip.vehicleId	
Vehicle	VehicleDayStatus.vehicleId	workshop days
Plan	Trip.planId	cascade delete
Plan	Deferral.planId	optional
Trip	TripStop.tripId	cascade delete
Trip	LoadCheck.tripId	cascade delete
Order	OrderLine.orderId	cascade delete
Order	Deferral.orderId	
Order	LoadCheck.orderId	
TripStop	DeliveryEvent.stopId	

Many-to-many

None are declared. An order reaches a trip through `TripStop` (Trip 1—* TripStop, Order 1—0..1 TripStop), which behaves as a join table that allows each order on at most one trip.

Logical references without a foreign key

These columns hold an id but the schema declares no relation, so the database does not enforce them:

`PlanVersion.planId`, `PlanAck.planId / userId`, `LoadMove.planId / orderId`, `LoadCheck.lineId / byUserId`, `Deferral.byUserId`, `Trip.loaderId`, `VehicleDayStatus.setById`, `Notification.outletId / userId`. The older `docs/data-model.md` draws `PlanVersion`, `PlanAck` and `LoadMove` as relations of `Plan`; in the Prisma schema they are logical references (dashed in the ER diagram).

9.6 Important constraints

Constraint	Model	Effect
<code>@unique</code> email	User	One account per email address
<code>@unique</code> driverId	Vehicle	One driver per vehicle and never two vehicles for one driver (also guaranteed by the admin API)
<code>@unique</code> orderId	TripStop, Receipt	An order has at most one stop and one receipt
<code>@@unique([date, depot])</code>	Plan	One plan per depot per day
<code>@@unique([planId, vehicleId, tripNo])</code>	Trip	No duplicate trip numbers for a vehicle in a plan
<code>@@unique([planId, version])</code>	PlanVersion	One snapshot per published version
<code>@@unique([planId, version, userId])</code>	PlanAck	A user acknowledges a version once
<code>@@unique([planId, version, orderId])</code>	LoadMove	An order is marked moved once per version
<code>@id([vehicleId, date])</code>	VehicleDayStatus	One workshop status per vehicle per day
<code>@id</code> eventId	DeliveryEvent	Idempotent sync of offline events
<code>onDelete: Cascade</code>	OrderLine→Order, Trip→Plan, TripStop→Trip, LoadCheck→Trip	Deleting a parent removes its children (used when demo data is reset)

9.7 Important indexes

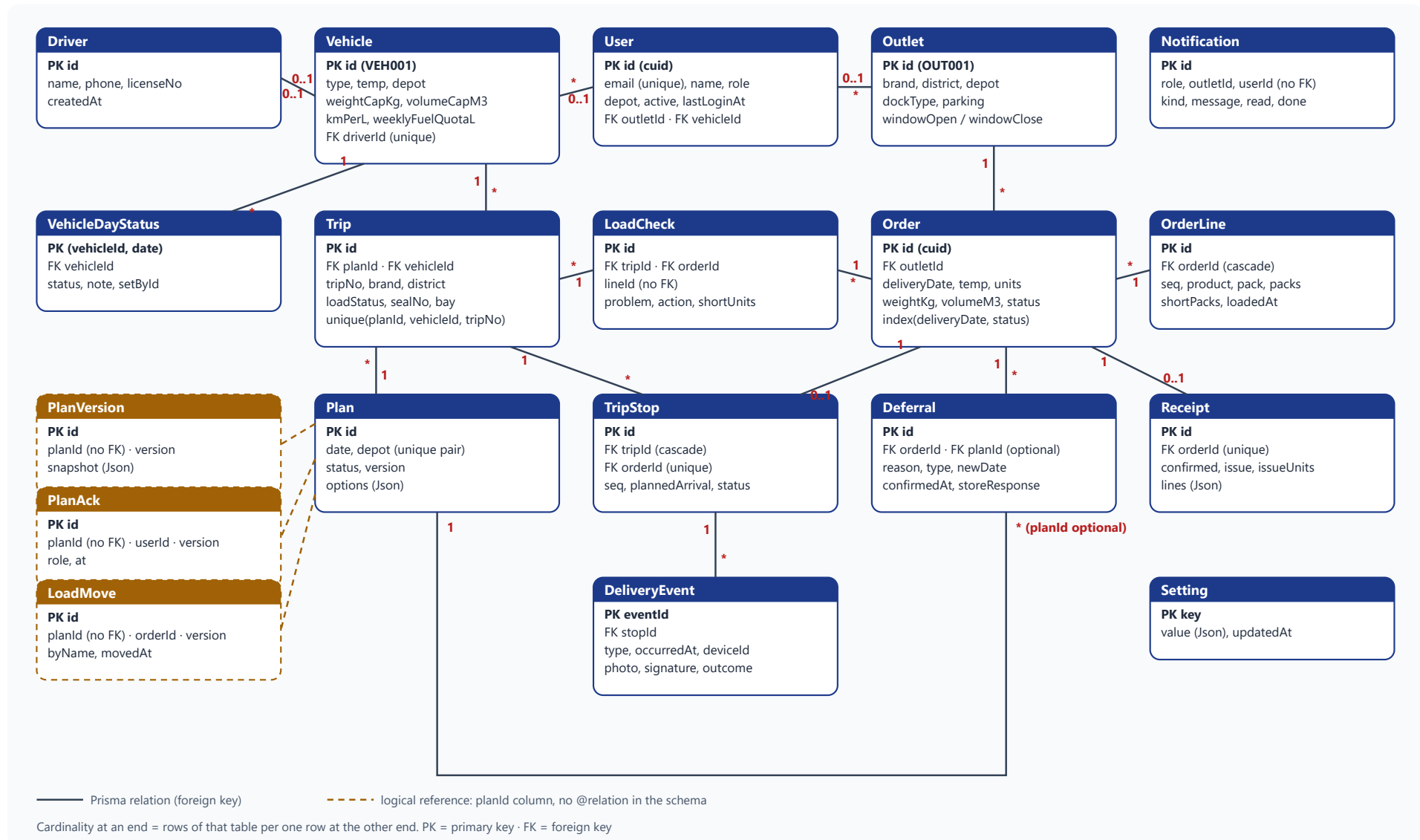
Index	Model	Supports
<code>(deliveryDate, status)</code>	Order	Loading a delivery day's orders by status
<code>(orderId)</code>	OrderLine, Deferral	Fetching lines and deferrals of an order
<code>(outletId, read)</code>	Notification	Unread count for a store
<code>(role, createdAt)</code>	Notification	Dispatcher alert feed
Unique indexes above		Each <code>@unique</code> / <code>@@unique</code> creates an index

9.8 Database business rules

- Orders are never split: `TripStop.orderId` is unique.

- A plan is per depot and per date; loaders and drivers use only published plans (`version > 0`).
- Every deferral stores a reason (`DeferralReason`) and a type (`unavoidable` or `choice`).
- A sealed or departed trip is identified by `sealedAt` / `departedAt` and is locked by the API (not by a database constraint).
- Packs on an order's lines add up to `Order.units` : an application rule (`goods.ts` , tested), not a database constraint.
- Each account has its own password hash; `active=false` switches the account off; `customPassword=true` stops the seed from resetting an admin-set password.
- The database stays in UTC; dates used for planning are `@db.Date` values interpreted as Sri Lanka calendar days by the application.

10. ER diagram



Entity-relationship diagram drawn only from `prisma/schema.prisma`: 19 models. Solid lines are Prisma relations; dashed lines are logical planId references without a relation.

11. Authentication and authorisation

11.1 Authentication architecture

- **Sign-in:** `POST /api/auth/login` with email and password returns `{ token, user }`. The token is a JWT signed with `JWT_SECRET`, valid for 12 hours, whose payload is the session user: `id, name, role, depot, outletId, vehicleId`.
- **Password storage:** `User.passwordHash` holds a bcrypt hash. Seeded accounts use passwords derived from `ACCOUNT_SECRET` (HMAC of the email, 12 characters), recomputed by the seed on every start unless an administrator set a custom password (`customPassword`).
- **Authorisation:** `requireRole(...roles)` verifies the token, loads `User.active` (a switched-off account gets 401 at its next request) and answers 403 when the role is not allowed.
- **Event streams:** browsers cannot send headers to `EventSource`, so `/api/changes` and `/api/events` take the token in the query string; `/api/changes` also checks the account is active.
- **Session in the browser:** kept in `localStorage` by `src/api.ts`. The driver app can add a local fingerprint / Face ID app lock (WebAuthn); it is not server authentication.
- **Admin actions:** create staff accounts and reset passwords (a random password is generated and shown once, bcrypt-hashed), switch accounts on and off; an administrator cannot switch themselves off and the last active administrator cannot be switched off.

Observed deviation from the documented design

The login handler in `apps/api/src/routes/auth.ts` contains:

```
const isDemoPw = body.data.password === 'waypoint-demo' || ... ||
body.data.password.trim().length > 0;
const valid = isDemoPw || (await bcrypt.compare(body.data.password, user.passwordHash));
```

Because the last operand is true for any non-empty string, the bcrypt comparison is never decisive: any non-empty password is accepted for an existing account. The handler also maps `dispatcher@`, `loader@`, `driver@` and `store@` aliases to specific accounts and falls back to the first user of a role when the email prefix matches. The README and the AI disclosure describe unique per-account passwords, so the documented behaviour and the source differ. This is reported in the Testing document as a finding.

11.2 Role-based access control

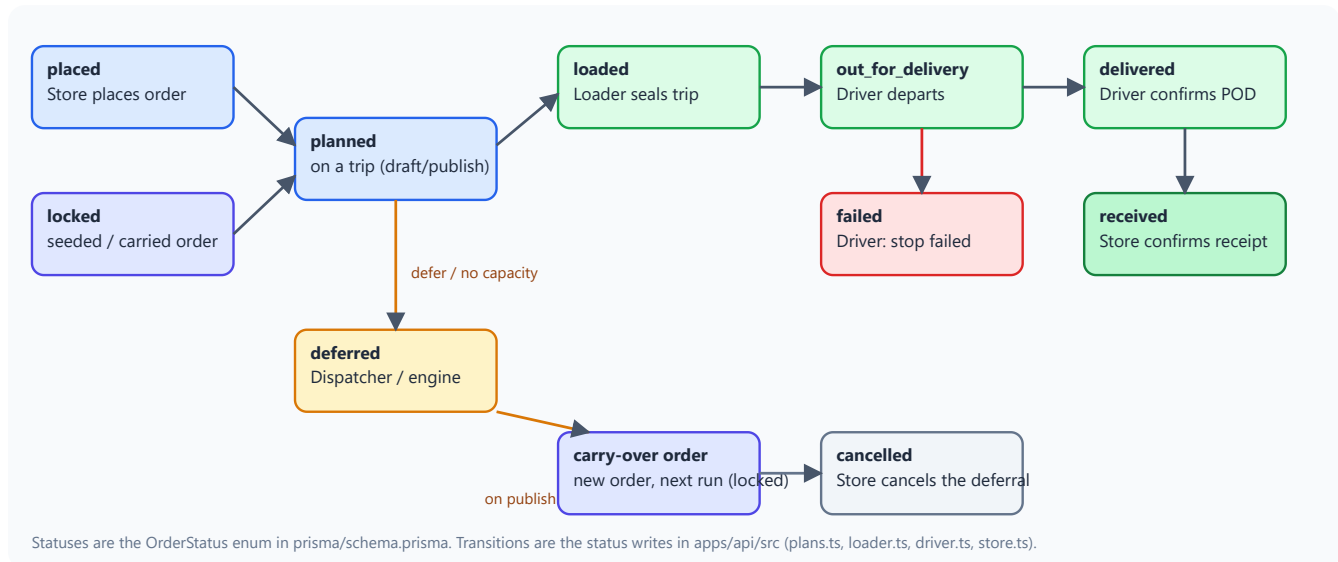
Role	API guard	Data scope	UI guard
dispatcher	<code>requireRole('dispatcher')</code>	An account with a depot is locked to it; the account without one (<code>dispatcher@waypoint.lk</code>) chooses Peliyagoda or Kandy with the switcher	Guard <code>role="dispatcher"</code> on <code>/dispatcher/*</code>
loader	<code>requireRole('loader')</code>	The loader's depot; <code>findTrip</code> refuses a trip of another depot ("not on your depot plan")	<code>/loader/*</code>
driver	<code>requireRole('driver')</code>	The vehicle in the token (<code>vehicleId</code>) and its depot	<code>/driver/*</code>

Role	API guard	Data scope	UI guard
store	<code>requireRole('store')</code>	The outlet in the token (<code>outletId</code>); every <code>/store/*</code> route is scoped to it	<code>/store/*</code>
admin	<code>requireRole('admin')</code>	All accounts, drivers, system and demo	<code>/admin/*</code>

A request from a signed-in account of another role answers **403** ("This action is for ... accounts"); a missing or expired token answers **401**; a switched-off account answers 401 at its next request. The web guard additionally redirects a user who opens another role's URL to their own area.

12. Order lifecycle

12.1 Order lifecycle



Order statuses and who moves an order between them.

Status	Set when	By
placed	A store places or replaces an order (<code>POST /api/store/orders</code>)	Store manager
locked	Seeded scenario order; carry-over order created at publish or seal	Seed / API
planned	The order is placed on a trip	Dispatcher / engine
deferred	Suggest plan or "defer" could not or chose not to serve it	Dispatcher / engine
loaded	The trip is sealed	Loader
out_for_delivery	The driver confirms the load and departs	Driver
delivered / failed	Stop delivered or failed (via <code>/api/sync</code>)	Driver
received	Store confirms receipt	Store manager
cancelled	Store cancels a deferred order's new-day delivery	Store manager

13. Deployment architecture

13.1 Docker architecture

`docker-compose.yml` defines four services and two named volumes (`pgdata` , `caddy_data`):

Service	Image / build	Ports	Notes
db	<code>postgres:16</code>	<code>127.0.0.1:\${DB_HOST_PORT:-15432}</code> (this machine only)	Database <code>waypoint</code> ; password <code>\${DB_PASSWORD:-waypoint}</code> ; health check with <code>pg_isready</code> ; stays on UTC
api	build <code>apps/api/Dockerfile</code> (<code>node:20-slim</code>)	internal 3000	Depends on a healthy db; environment: <code>DATABASE_URL</code> , <code>JWT_SECRET</code> , <code>ACCOUNT_SECRET</code> , <code>DEMO_*</code> , <code>API_PORT</code> , <code>TZ=Asia/Colombo</code> ; <code>restart: unless-stopped</code>
web	build <code>apps/web/Dockerfile</code> (build stage <code>node:20-slim</code> , runtime <code>nginx:1.27-alpine</code>)	internal 80	Serves the Vite build; files made world-readable so <code>/sw.js</code> does not answer 403
caddy	<code>caddy:2</code>	80, 443	<code>Caddyfile</code> mounted read-only; <code>DOMAIN</code> defaults to <code>localhost</code> ; certificate data in <code>caddy_data</code>

API start-up command (`apps/api/Dockerfile`):

```
pnpm prisma db execute --schema prisma/schema.prisma --file prisma/pre-push.sql \
&& pnpm prisma db push --skip-generate \
&& pnpm db:seed \
&& pnpm --filter @waypoint/api start
```

The image copies `prisma/` , `data/` , `packages/shared` , `apps/api` and `scripts/credentials.ts` , so `pnpm credentials` works inside the container. `.dockerignore` keeps `node_modules` , `dist` , `.git` , `.env` and `credentials` out of the images.

Local override file

The untracked file `docker-compose.override.yml` in the working copy additionally publishes PostgreSQL on host port 5432 so an API running outside Docker can connect. It is a local development convenience, not part of the tracked deployment.

13.2 Deployment architecture

- **Single host:** the whole system is one `docker compose up` on one Linux server. Only Caddy publishes public ports (80/443).
- **HTTPS:** Caddy obtains a certificate for `DOMAIN` on the first request (`docs/deploy.md` names Let's Encrypt). Without a domain, `<ip-with-dashes>.sslip.io` is used.

- **One-command install:** `scripts/server-setup.sh` (per `docs/deploy.md` and `docs/handoff.md`) installs Docker, adds swap, clones the repository to `/opt/waypoint`, writes `.env` with random secrets, runs `docker compose up -d --build` and waits for `/api/health`.
- **Live instance:** <https://3-106-77-179.sslip.io>. The AI disclosure records a deployment to AWS Lightsail with images built on a Mac for x86 because the 512 MB server cannot build them. Reachable (HTTP 200) on 2026-10-04.
- **Android app:** built by the manual GitHub Actions workflow `android-app.yml` (Bubblewrap, Trusted Web Activity) and published as `/downloads/waypoint-connect.apk` with `/.well-known/assetlinks.json`. The APK is a wrapper around the live site.
- **Team database access:** through an SSH tunnel to `127.0.0.1:15432` on the server; the port is never open to the internet.

14. Setup instructions

14.1 Prerequisites

Item	Requirement	Check
Node.js	20 or newer (<code>engines</code>); Docker images use Node 20	<code>node -v</code>
pnpm	9.12.0 via Corepack (<code>packageManager</code>)	<code>corepack enable</code> then <code>pnpm -v</code>
Docker	Docker with Compose v2	<code>docker compose version</code>
PostgreSQL	16 (Docker <code>db</code> service) or a local server	see Database setup
Python 3 + pandas	Only for <code>pnpm check:2b</code>	
Git	To clone the repository	

14.2 Environment configuration

Copy the example and change every `CHANGE_ME` value. Docker Compose reads `.env` in the project folder; `pnpm dev:api` reads it through `--env-file=../../.env`. `.env` is gitignored; never commit it.

```
cp .env.example .env          # Windows PowerShell: Copy-Item .env.example .env
```

Variable	Meaning	Default / note
<code>DOMAIN</code>	Public domain for Caddy (no https://)	<code>localhost</code>
<code>DB_PASSWORD</code>	PostgreSQL password	compose default <code>waypoint</code> . Must equal the password inside <code>DATABASE_URL</code>
<code>DATABASE_URL</code>	Connection string for local (non-Docker) API / Prisma	<code>postgresql://postgres:CHANGE_ME@localhost:15432/waypoint</code>
<code>JWT_SECRET</code>	Signs the 12-hour session tokens	compose fallback <code>dev-only-secret-change-me</code> : always change on a server
<code>ACCOUNT_SECRET</code>	Derives every account password	falls back to <code>JWT_SECRET</code> ; changing it changes all passwords
<code>API_PORT</code>	API port	3000
<code>DEMO_DATE</code>	Pin all screens to one day (YYYY-MM-DD)	empty = real date
<code>DEMO_MODE</code>	Enables the demo-only simulate route	compose default 1
<code>DEMO_CLOCK</code> , <code>DEMO_LOADER_CLOCK</code> , <code>DEMO_STORE_CLOCK</code>	Pin a role's clock (HH:MM)	empty = real Colombo time
<code>DB_HOST_PORT</code>	Host port for the database	15432
<code>LOG_LEVEL</code>	API log level (read by the API)	info

14.3 Repository, install, Prisma, database, seed

```
git clone https://github.com/Cognix-0/DarkHorse_WaypointConnect.git
cd DarkHorse_WaypointConnect
corepack enable
pnpm install
cp .env.example .env          # then edit the CHANGE_ME values
docker compose up -d db       # PostgreSQL on 127.0.0.1:15432 (or use any local PostgreSQL and
set DATABASE_URL)
pnpm prisma generate
pnpm prisma db push           # creates the tables from prisma/schema.prisma
pnpm db:seed                  # outlets, vehicles, drivers, 188 accounts, today's orders
```

The scripts are from the root `package.json`: `db:seed = tsx prisma/seed.ts`; `credentials = tsx scripts/credentials.ts`. The seed is safe to run repeatedly. It needs `ACCOUNT_SECRET` or `JWT_SECRET` in the environment (otherwise it stops with "Set ACCOUNT_SECRET..."). A `db:migrate` script (`prisma migrate dev`) exists, but Docker and the README use `db push`.

14.4 Running the API and the web app

```
pnpm dev:api    # http://localhost:3000/api/health
pnpm dev:web    # http://localhost:5173 (proxies /api to localhost:3000)
```

Print the account passwords with `pnpm credentials` (output is CSV: role, depot, vehicle_or_store, name, email, password). Keep it private.

14.5 Tests and checks

```
pnpm test      # 21 planning-engine tests
pnpm typecheck # all three workspaces (run pnpm prisma generate first)
pnpm check:2b  # engine on the official peak day + check_allocation.py (needs Python 3 + pandas)
```

14.6 Production build

```
pnpm --filter @waypoint/web build # tsc + vite build into apps/web/dist
```

The API is run from TypeScript with `tsx` (`pnpm --filter @waypoint/api start`); there is no separate API build step.

14.7 Running with Docker Compose

```
cp .env.example .env
docker compose up --build          # database + API (schema + seed) + web + HTTPS
docker compose up -d --build      # detached; rebuild after code changes
docker compose logs --tail=50 api # wait for the seed and "Server listening"
docker compose down -v && docker compose up -d --build # wipe the database and re-seed
```

Open <https://localhost> and accept the local certificate once. Only Caddy publishes ports 80 and 443. After `git pull`, rebuild with `docker compose up -d --build`; on phones close and reopen the app once.

14.8 Deployment

From [docs/deploy.md](#). Server: Ubuntu 24.04, 2 vCPU, 2 GB RAM, 20 GB disk, ports 22/80/443 open.

One command

```
curl -fsSL https://raw.githubusercontent.com/Cognix-0/DarkHorse_WaypointConnect/main/scripts/server-setup.sh | bash
# with a domain: curl ... | DOMAIN=waypoint.example.com bash
```

It prints `Live: https://...` after 5–10 minutes; without a domain it uses `<ip-with-dashes>.sslip.io`. Run it again to deploy a new commit.

By hand

1. Point an A record at the server (`dig +short your.domain` must print the IP).
2. Install Docker: `curl -fsSL https://get.docker.com | sudo sh`.
3. Clone, `cp .env.example .env`, set `DOMAIN`, `DB_PASSWORD`, `JWT_SECRET`, `ACCOUNT_SECRET` (e.g. `openssl rand -hex 32`).
4. `docker compose up -d --build` then `docker compose logs -f api`.
5. List passwords: `docker compose exec api pnpm -s credentials > credentials.csv` (keep private).

Checklist before submitting: padlock on the HTTPS URL; walkthrough accounts sign in; Order Queue shows 85 orders; driver app loads offline on a phone. Live deployment recorded in this project: <https://3-106-77-179.sslip.io>.

14.9 Troubleshooting and common errors

Symptom	Cause and fix	Source
<code>TS2339: Property 'driver' does not exist on type PrismaClient</code> (and similar errors for <code>active</code> , <code>lastLoginAt</code>) in <code>pnpm typecheck</code>	The generated Prisma client is older than the schema. Run <code>pnpm prisma generate</code> , then typecheck again.	observed 2026-10-04; CI runs generate first
<code>pnpm: not found: node</code>	Node.js is not on the PATH of the shell. Install Node 20+ and reopen the terminal.	pnpm-error.txt

Symptom	Cause and fix	Source
<code>--progress</code> is a global <code>compose</code> flag	Put the flag before the subcommand: <code>docker compose --progress=plain build web</code> .	docker-build.txt
Seed stops with "Set ACCOUNT_SECRET (or JWT_SECRET)..."	Provide the secret in the shell environment or <code>.env</code> .	prisma/accounts.ts
Browser shows ERR_TOO_MANY_REDIRECTS	The database container could not bind its host port, so db, api and web did not start. Run <code>docker compose ps</code> ; set <code>DB_HOST_PORT</code> .	docs/handoff.md
"Wrong email or password" for a seeded account	Passwords come from <code>ACCOUNT_SECRET</code> ; changing it (or <code>JWT_SECRET</code> when empty) changes all passwords. Re-run <code>pnpm credentials</code> .	docs/deploy.md
Certificate error / Caddy "challenge failed"	DNS not pointing at the server yet, or ports 80/443 closed. Fix, then <code>docker compose restart caddy</code> .	deploy.md
Build stops with "killed"	Out of memory: add 2 GB swap.	deploy.md
New screens missing after pull	Rebuild with <code>--build</code> ; close and reopen the phone app (service worker).	deploy.md , handoff.md
<code>/sw.js</code> answers 403	Fixed in apps/web/Dockerfile (files made world-readable); rebuild the web image.	ai-disclosure 2026-10-04
Store or driver account shows the wrong store/vehicle	Database seeded by an older version: <code>docker compose down -v && docker compose up -d</code> .	deploy.md

15. Testing

15.1 Testing strategy

The project has one automated test suite: the planning engine and goods catalogue in `packages/shared/test` (Node test runner, 21 tests). Everything else is verified by typecheck, build, the official allocation checker, and end-to-end runs that the team recorded in `docs/ai-disclosure.md`. Those recorded runs are labelled **REPORTED** below: they are stated in the project files and were **not** re-executed for this report. Checks executed on **2026-10-04** while generating this report are labelled **PASS**.

Category	Present in repository?	Detail
Unit tests	Yes	21 tests in three files (engine 13, goods 3, schedule 5)
Integration / API tests	No test files	API flows were exercised by scripts during development (reported in the disclosure); the scripts are not stored in the repository
Frontend tests	No	No test files in <code>apps/web</code> ; screens were checked in headless Chromium (reported)
Type checking	Yes	<code>pnpm typecheck</code> (tsc in shared, api, web)
Build verification	Yes	<code>pnpm --filter @waypoint/web build</code> ; Docker image builds in CI
CI	Defined	<code>.github/workflows/ci.yml</code> : test, prisma generate, typecheck, check:2b, docker compose build. Run results are <i>Not verified from the provided project files.</i>
Docker / database	Reported	Full stack on real PostgreSQL in Docker and against the live URL (disclosure 2026-10-04). <code>docker compose config</code> validated on 2026-10-04

15.2 Executed on 2026-10-04 (Windows 11, Node v24.21.0, pnpm 9.12.0)

Test ID	Test	Expected result	Actual result	Status
V-01	<code>pnpm test</code>	All engine tests pass	tests 21, pass 21, fail 0, duration about 0.7 s	PASS
V-02	<code>pnpm typecheck</code> before <code>prisma generate</code>	No errors	apps/api: 11 TS errors (stale Prisma client: <code>driver</code> , <code>active</code> , <code>lastLoginAt</code> , role <code>admin</code>)	FINDING
V-03	<code>pnpm prisma generate</code> then <code>pnpm typecheck</code>	No errors in shared, api, web	All three workspaces: Done, exit code 0	PASS
V-04	<code>pnpm --filter @waypoint/web build</code> (via <code>apps/web build</code>)	tsc and Vite build succeed	134 modules transformed; JS 471.72 kB (135.27 kB gzip), CSS 46.43 kB; built in 4.25 s	PASS
V-05	Engine on the official peak day (<code>scripts/run-task2b.ts</code>)	Every order served or deferred with a reason	85 orders: 72 served on 32 trips, 13 deferred (all with a reason)	PASS
V-06	<code>docker compose config -q</code>	Compose file valid	Exit code 0	PASS

Test ID	Test	Expected result	Actual result	Status
V-07	Live URL https://3-106-77-179.sslip.io	HTTPS reachable	HTTP 200	PASS
V-08	Official checker check_allocation.py	FEASIBILITY: PASSED	Not run: Python is not installed on the verification machine. README and disclosure state PASSED	REPORTED

15.3 Automated test inventory (all passed in V-01)

Test ID	File	Test	Expected	Status
U-01	engine.test.ts	trip time matches the booklet example: Fresh Gampaha, 2 rear docks + 1 street = 101 min	Pass	PASS
U-02	engine.test.ts	trip time matches the booklet example: Fresh Colombo, 4 street stops = 112 min	Pass	PASS
U-03	engine.test.ts	two Fresh trips of 101 + 112 min fit the 270 min window; a third trip is refused	Pass	PASS
U-04	engine.test.ts	chilled goods need a refrigerated vehicle	Pass	PASS
U-05	engine.test.ts	van-only outlets cannot be served by trucks	Pass	PASS
U-06	engine.test.ts	a vehicle only serves its home depot	Pass	PASS
U-07	engine.test.ts	vehicles in the workshop are refused	Pass	PASS
U-08	engine.test.ts	weight and volume caps are both enforced, with a readable reason	Pass	PASS
U-09	engine.test.ts	one brand and one district per trip	Pass	PASS
U-10	engine.test.ts	Fresh trips must fit the 270 min pre-dawn window	Pass	PASS
U-11	engine.test.ts	weekly fuel quota is enforced when fuel use is known	Pass	PASS
U-12	engine.test.ts	priority: skipped-yesterday first, then Fresh chilled, Fresh ambient, Tech, Style	Pass	PASS
U-13	engine.test.ts	suggest plan on the official peak day (S1): every served trip is valid and every order has a decision	Pass	PASS
U-14	goods.test.ts	goods lines: packs always add up to the order units	Pass	PASS
U-15	goods.test.ts	goods lines are stable and follow the brand and temperature	Pass	PASS
U-16	goods.test.ts	store orders: weight and volume come from the pack, units are the packs	Pass	PASS
U-17	schedule.test.ts	ETAs follow the planning standard: outbound, then handling + inter-stop per stop	Pass	PASS
U-18	schedule.test.ts	a single Fresh trip waits so it reaches the first shop as it opens	Pass	PASS
U-19	schedule.test.ts	the second trip starts when the first ends (planning standard); the first leaves at 03:30	Pass	PASS
U-20	schedule.test.ts	stops are ordered by closing time and late arrivals are flagged	Pass	PASS

Test ID	File	Test	Expected	Status
U-21	schedule.test.ts	S1 suggested plan: every trip gets a timetable that matches its trip minutes	Pass	PASS

15.4 Recorded in the project files (not re-executed here)

Test ID	Test	Expected result	Actual result (as recorded)	Status
R-01	Full stack on real PostgreSQL in Docker; every endpoint called over HTTPS and validated against <code>contract.ts</code>	All responses match the contract	Passed for dispatcher suggest/publish, loader claim/tick/problem/seal, driver sync incl. idempotent replay, POD, store order (ai-disclosure 2026-10-04)	REPORTED
R-02	Sign-in of all 188 accounts with own passwords; old logins rejected	Each account signs in with its own password	Recorded as passed. Conflicts with the source finding F-01 below.	REPORTED
R-03	Role isolation and depot lock	Dispatchers cannot open the other depot; non-admin blocked from admin routes	Recorded as passed (disclosure rows for personal accounts and admin console)	REPORTED
R-04	Admin actions kept after API restart	Assignments, switch-offs, resets and new accounts persist	Recorded as passed; screens checked in headless Chrome, no console errors	REPORTED
R-05	Next-run order cycle at 10:00 and 18:00 for both depots	Store order listed for the next run, then planned	Recorded as passed; next-day build tested twice (idempotent)	REPORTED
R-06	Offline driver flow: real offline reload via service worker; duplicate-safe resend	App opens offline; repeated event delivered once	Recorded as passed in Chromium (disclosure 2026-10-02)	REPORTED
R-07	Real-time refresh	Signal about 260 ms after an order; a new store order on an open dispatcher queue in 630 ms	Recorded as passed	REPORTED
R-08	Demo mode story rehearsed twice end to end	Every step passes; End demo returns to real time	Recorded as passed; plan identical each reset (OUT026 on VEH031 trip 2, 10 unavoidable deferrals)	REPORTED
R-09	Driver fingerprint unlock	Turn on, lock on reopen, wrong finger rejected, unlock, turn off	Recorded as passed with a virtual authenticator	REPORTED

15.5 Authentication and authorisation testing

Test ID	Test	Expected result	Actual result	Status
F-01	Source inspection of <code>POST /api/auth/login</code> (<code>apps/api/src/routes/auth.ts</code>)	Only the correct password for the account is accepted	The check <code>isDemoPw = ... password.trim().length > 0</code> is true for any non-empty password, so <code>valid</code> is always true for an existing account. Aliases (<code>dispatcher@</code> , <code>loader@</code> , <code>driver@</code> , <code>store@</code>) and a first-user-of-role fallback also exist. Not executed against a running server; the logic is direct from the code	FINDING
F-02	Source inspection of <code>requireRole</code>	401 without token, 401 for a switched-off account, 403 for another role	Implemented as expected in <code>auth.ts</code> (every route group uses it)	PASS
F-03	Source inspection: depot and outlet scoping	Dispatcher with depot locked; store routes scoped to <code>outletId</code> ; loader refuses other depot trips	Implemented in <code>dayOf()</code> , <code>ctxOf()</code> , <code>findTrip()</code>	PASS
F-04	Repository secrets	No password files in the repository	<code>data/user_credentials.csv</code> is tracked in git and contains per-account passwords. Its values are not reproduced in this report	FINDING

Recommended before relying on the system

- Remove the `|| password.trim().length > 0` operand and the alias / fallback logic from `auth.ts` .
- Remove `data/user_credentials.csv` from the repository and rotate `ACCOUNT_SECRET` / reset passwords, because its contents are in git history.
- Regenerate these documents afterwards so they describe the corrected behaviour.

15.6 Business-rule testing

Rule	Covered by	Status
Chilled needs reefer; van-only; home depot; workshop; weight/volume; one brand + district; fuel quota	U-04 to U-11	PASS
Trip minutes (booklet examples) and Fresh 270-minute window; max 2 trips	U-01 to U-03, U-10	PASS
Priority order	U-12	PASS
Suggest plan: every order has a decision; every served trip valid	U-13, U-21, V-05	PASS
Goods packs add up to units	U-14 to U-16	PASS
Timetable: ETAs, second trip start, closing-time order	U-17 to U-21	PASS
16:00 cutoff, current run, next operating day, Colombo time	No automated test; reported in R-05	REPORTED

Rule	Covered by	Status
Sealed / departed trips locked; second-deferral confirmation; driver one-vehicle rule; idempotent sync	No automated test in the repository; reported in R-01, R-04, R-06	REPORTED

15.7 Gaps

- No automated API, frontend or end-to-end tests are stored in the repository.
- CI results are *Not verified from the provided project files.*
- Docker build and run, and the official checker, were not re-executed for this report.

16. Judge walkthrough

16.1 Before you start

- **Base URL:** the live system is <https://3-106-77-179.sslip.io>. A local Docker run is <https://localhost> (accept the local certificate once). Paths below are appended to the base URL.
- **Passwords:** every account has its own password, supplied with the submission (see [pnpm credentials](#)). Passwords are not printed in this guide.
- **Windows:** use a separate browser window (or private window) for each role so all stay signed in side by side (advice shown on the Demo mode page). Use a laptop for the dispatcher and loader, and a phone or Chrome DevTools device mode for the driver and the store manager.
- **Time:** about 15 minutes.

Two ways to run the walkthrough

Real time (default): every screen uses the real Asia/Colombo date and time; each day contains the 85 Peliyagoda orders of the official peak-day scenario.

Demo mode (recommended for judges): the administrator starts Demo mode (Step 1) and the whole system runs on the peak day **Wed 25 Mar 2026**: the store manager is at Tue 24 Mar 14:18 (before the cutoff), the loader at 02:40 and the dispatcher at 06:42. The story below follows it.

16.2 Story in one line

A store orders → the dispatcher plans the peak day, handles the chilled overload and publishes → the loader loads and seals → the driver delivers (partly offline) → the store confirms receipt with a shortage → the dispatcher sees the alert and the proof of delivery.

16.3 Steps

STEP 1 · Open the application and start Demo mode		Role: Administrator
URL	https://3-106-77-179.sslip.io/login , then /admin/demo	
Login	admin@waypoint.lk (password supplied with the submission)	
Expected screen	Sign-in page ("One platform from order to delivery", the four steps Order · Plan · Load · Deliver, 120 outlets · 60 vehicles · 2 depots). After sign-in the Admin Console opens on System ("All systems running").	
Action	Open Demo mode in the sidebar. Choose a length (default 2 h) and press Start demo .	
Expected result	A confirmation "Demo mode on". The page shows the three role clocks (Tue 24 Mar · 14:18 store, Wed 25 Mar · 02:40 loader, Wed 25 Mar · 06:42 dispatcher) and a live checklist that follows the story: order → plan → deferrals and store answer → loading → both trips → receipt issue → reply. Keep this page open: it names the vehicle and driver account that carry the store's order.	

STEP 2 · Store manager: order, status and information

Role: Store manager

URL	/store (Today), /store/order (Place order), /store/orders and /store/orders/:id
Login	out026@waypoint.lk (OUT026, Waypoint Fresh, Gampaha; phone layout or DevTools phone mode)
Expected screen	Today: cutoff countdown (before 16:00), today's deliveries, tomorrow's order cards for dry and chilled, this week, recent activity. In Demo mode OUT026's dry order is deliberately missing so it is placed live.
Action	Place order → <i>Dry groceries</i> → keep the suggested packs (or use <i>Copy last order</i>) → Place order . Then open Orders and the new order.
Expected result	The order is accepted before the 16:00 cutoff, so it goes on the next run and can be edited until 16:00 (a notification says so). The order page shows the status steps Received → Scheduled → Loaded → Delivered, the lines in packs and the weight. The same order appears at once on the dispatcher's screens without a refresh.

STEP 3 · Dispatcher: plan, defer, publish

Role: Dispatcher

URL	/dispatcher , /dispatcher/queue , /dispatcher/board , /dispatcher/deferrals , /dispatcher/live , /dispatcher/forecast
Login	dispatcher@waypoint.lk (one account for both depots; use the depot switcher, start with Peliyagoda)
Expected screen	Overview: about 85 orders (84 scenario orders plus the store's live order), reefer space far over capacity (README: 193 %), "Needs your attention".
Action	1) Order Queue: shops skipped on the last run are red at the top; orders placed after the cutoff are greyed out. Press Auto-allocate & open board. 2) Planning Board: choose <i>All available</i>, drag a stop marked <i>chilled</i> onto a Dry truck. 3) Deferrals: read the "Payday overload" banner and the deferral table; try recovery options A–C; press Confirm & publish plan.
Expected result	Auto-allocate shows "All booklet rules pass" (README quotes 75 of 85 orders on 34 trips in the app; the engine alone gives 72 of 85 on 32 trips, which was reproduced). The chilled drop shows Drop blocked with the exact rule ("... can't carry chilled goods") and the vehicles that would take it. Every deferred order has a reason and the label <i>unavoidable</i> or <i>choice</i> . After publishing, every affected store is notified.
Depot restriction	This dispatcher account switches depots. A dispatcher account created by the administrator <i>with a depot</i> (Step 6) can only see that depot: the API ignores the <code>?depot=</code> request for such accounts.

STEP 4 · Loader: load, report a problem, seal

Role: Loader

URL	<code>/loader</code> , <code>/loader/trip/:tripId</code> , <code>/loader/changes</code>
Login	loader1.peliyagoda@waypoint.lk (laptop or 1280 × 800 tablet size)
Expected screen	Choose vehicle: every vehicle on the published plan as a card (bay, departure, items loaded, who is loading). Nothing appears until the dispatcher has published.
Action	Open the vehicle that the Demo mode page names for the store's order (README example: VEH024). Trip 1: tick every line → Seal & release (type any seal number; reefer vehicles also need a temperature). Trip 2 (the store's stop): on the first line press ! → <i>Damaged goods, 2 packs, Load the rest, send the missing packs tomorrow</i> → Send report ; tick the rest → Seal & release .
Expected result	Sealing is refused while any line is neither ticked nor reported. The 2 damaged packs become an order on the next run. The dispatcher's Live Tracking shows a <i>Loader flag</i> alert; the store is told; the driver's manifest shows the short count. Plan changes lists what moved between plan versions (appears after a republish).

STEP 5 · Driver: assigned vehicle, route, delivery, offline

Role: Driver (vehicle account)

URL	<code>/driver</code> , <code>/driver/load/:tripId</code> , <code>/driver/run</code> , <code>/driver/stop/:id</code> , <code>/driver/sync</code>
Login	The vehicle account named on the Demo mode page (README example: veh024@waypoint.lk, driver Nimal Fernando). Use a phone or DevTools phone mode (390 × 844).
Expected screen	Today's runs: the vehicle, its assigned driver, the trips with load status and fuel.
Action	Review load & start Trip 1 → Confirm load & depart (enabled only once the loader has sealed) → I've arrived → Start unloading → photo and signature → Confirm delivery . Then go offline (airplane mode, or DevTools → Network → <i>Offline</i>) and reload: the app still opens. Deliver the next stop or report a problem; the Sync tab lists what waits. Go online again.
Expected result	Status updates reach the dispatcher's Live Tracking. Offline actions are queued with a UUID and sent in order on reconnect; the Back online sheet lists what was sent and any plan change meanwhile; a repeated send never delivers twice. On Trip 2 the store's stop shows the short packs from the dock.

STEP 5b · Close the loop

Roles: Store manager, then Dispatcher

URL	<code>/store/orders/:id/receipt</code> , then <code>/dispatcher/live</code>
Action	Store manager: Orders → the delivered order (photo and signature from the driver) → Confirm receipt , mark one line <i>Short</i> → Confirm receipt · 1 issue . Dispatcher: open Live Tracking , answer the alert (<i>Reply to store</i>) and open View POD .
Expected result	A <i>Store issue</i> alert and a <i>POD</i> alert appear on Live Tracking; View POD shows the driver's photo and signature. The Demo mode checklist ticks the remaining steps.

STEP 6 · Administrator: System, Vehicles & drivers, Accounts

Role: Administrator

URL	<code>/admin</code> , <code>/admin/fleet</code> , <code>/admin/accounts</code> , <code>/admin/demo</code>
Login	admin@waypoint.lk
Expected screen	System: database and API status, the working day and clock every screen uses, orders, vehicles, vehicles without a driver and today's plan per depot, plus <i>Build today's orders</i> .
Action	Vehicles & drivers: assign a driver to a vehicle; choosing a driver already on another vehicle opens <i>Move this driver?</i> and needs confirmation; Add driver . Accounts: search; switch an account on/off; Reset password (the new password is shown once); New staff account → Create account (dispatcher, loader or administrator, with a depot for loaders).
Expected result	A driver is never on two vehicles (database unique index). A switched-off account cannot sign in and an open session stops at its next request. The administrator cannot switch themselves off, and the last active administrator cannot be switched off. Changes stay after an API restart. Finish with End demo now on Demo mode to return every role to real Sri Lanka time.

STEP 7 · Business-day and cut-off logic

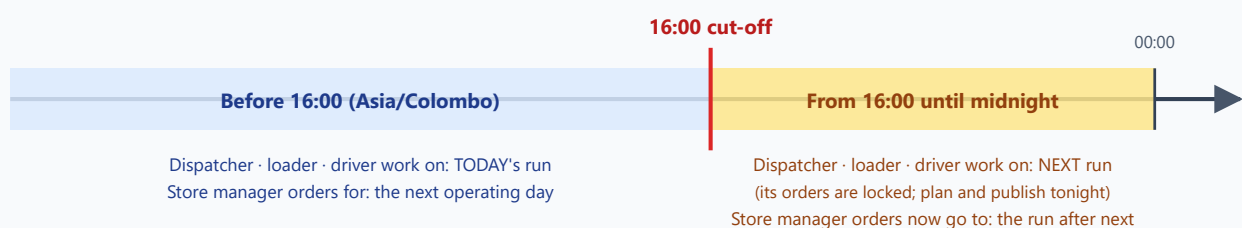
Roles: all

URL	<code>/store</code> , <code>/dispatcher/queue</code> , <code>/loader</code> , <code>/driver</code>
Action	Compare the screens before and after 16:00 Asia/Colombo (see the next section). In Demo mode the store clock shows 14:18 so the cutoff countdown is open.
Expected result	Before 16:00 the working run is today; after 16:00 it is the next operating day. See the table in the next section.

16.4 Business-day and cut-off logic

Source: apps/api/src/reference.ts (workingDay, ORDER_CUTOFF, nextOperatingDay) and apps/api/src/routes/store.ts (ctxOf)

"Next operating day" skips non-operating days in data/calendar.csv.



At midnight the calendar day rolls over and the day is built automatically on the first request (builtDays in server.ts).

Working run before and after the 16:00 cut-off (from reference.ts and store.ts).

Role	Before 16:00	From 16:00 (until midnight)
Store manager	Orders placed now are for the next operating day and can be changed until 16:00. Countdown to the cutoff is shown.	The cutoff has passed: an order placed now goes on the run after that ("Placed after the cutoff, so it goes on the ... run").
Dispatcher	Plans, tracks and publishes today's run. Orders already in for the next run appear greyed out ("next run").	Works on the next operating day : its orders are locked, so the dispatcher plans and publishes it in the evening.
Loader	Loads today's run (loading from 02:40).	Sees the next run's published plan.

Role	Before 16:00	From 16:00 (until midnight)
Driver	Delivers today's run (03:30–08:00 for Fresh).	Sees the next run's route once it is published.

DEMO_DATE and clocks

- `DEMO_DATE=YYYY-MM-DD` in `.env` pins every screen to one fixed day (the 16:00 switch is then off: `workingDay()` returns that day). Leave it empty for the real date.
- `DEMO_CLOCK`, `DEMO_LOADER_CLOCK` and `DEMO_STORE_CLOCK` pin the dispatcher, loader and store clocks (for example `DEMO_STORE_CLOCK=16:30` would show the store the closed cutoff; this combination was not executed while writing this document).
- **Admin** → **Demo mode** replaces this at run time with the peak day 2026-03-25 and the three clocks above, and restores real time when it ends.
- Daily build: each new day is built automatically on the first request after midnight; the administrator can also press *Build today's orders*.

17. Designathon requirements

17.1 Basis of the comparison

Original proposal not provided

The original Designathon proposal / Challenge Booklet was **not** among the provided files. The "original requirements" below are the ones recorded in the repository: the deliverables list in `docs/handoff.md`, the booklet rules in `CLAUDE.md` and `README.md`, and the Figma frames (D1–D6, L1–L5, M1–M8, SM1–SM9) that the README maps to screens. Anything outside these *Not verified from the provided project files.* . Statuses use the project files as evidence only.

17.2 Deliverables

Original requirement	Current implementation	Status	Evidence / location
Responsive web app for four roles	React PWA with dispatcher, loader, driver and store-manager areas; phone and desktop layouts for the store, tablet for the loader, phone for the driver	IMPLEMENTED	<code>apps/web/src/App.tsx</code> , <code>roles/*</code>
Public GitHub monorepo (branch main)	pnpm monorepo exists; the repository URL is recorded; its public visibility could not be checked	NOT VERIFIED	<code>docs/handoff.md</code> , <code>pnpm-workspace.yaml</code>
<code>docker compose up</code> works and seeds demo data	Four-service compose file; the API container syncs the schema and seeds on every start; the team records a successful run on real PostgreSQL (2026-10-04)	IMPLEMENTED	<code>docker-compose.yml</code> , <code>apps/api/Dockerfile</code> , ai-disclosure. <code>docker compose config</code> validates; a full build was not re-run for this document
Public HTTPS URL	<code>https://3-106-77-179.sslip.io</code> answers HTTP 200	IMPLEMENTED	<code>README.md</code> ; checked on 2026-10-04
Four seeded accounts	188 personal accounts (1 admin, 1 dispatcher, 6 loaders, 60 vehicle/driver, 120 store) with derived passwords	CHANGED	ai-disclosure 2026-10-04; <code>prisma/accounts.ts</code>
Numbered judge walkthrough in the README	16 numbered steps in five groups; parts are out of date (Simulate morning, vehicle and count details)	IMPLEMENTED	README "Judge walkthrough"
docs: architecture, data model, AI disclosure	All three exist; <code>docs/data-model.md</code> is not yet committed	IMPLEMENTED	<code>docs/</code>
5–8 minute video	A timed storyboard exists; a recorded video is not among the files	NOT VERIFIED	<code>docs/video-script.md</code> ; README checklist unticked

17.3 Booklet rules recorded in the repository

Original requirement	Current implementation	Status	Evidence / location
Chilled orders need a reefer vehicle	Rule <code>NEEDS_REEFER</code>	IMPLEMENTED	<code>engine/rules.ts</code> ; test "chilled goods need a refrigerated vehicle"
van_only outlets need a van	<code>NEEDS_VAN</code>	IMPLEMENTED	<code>rules.ts</code> ; test "van-only outlets cannot be served by trucks"
A vehicle serves only its home depot	<code>WRONG_DEPOT</code>	IMPLEMENTED	<code>rules.ts</code> ; test
Max 2 trips per vehicle per day	<code>MAX_TRIPS</code>	IMPLEMENTED	<code>rules.ts</code> ; test "...a third trip is refused"
One brand and one district per trip	<code>TRIP_MIXES_BRAND</code> , <code>TRIP_MIXES_DISTRICT</code>	IMPLEMENTED	<code>rules.ts</code> ; test
Orders are never split	One <code>TripStop</code> per order (unique <code>orderId</code>)	IMPLEMENTED	<code>schema.prisma</code>
Trip minutes formula	Implemented in the engine; the booklet examples (101 and 112 min) are tests	IMPLEMENTED	<code>engine/rules.ts</code> ; tests
Fresh window 270 min; Style/Tech 480 min	<code>OVER_TIME_FRESH</code> , <code>OVER_TIME_DAY</code>	IMPLEMENTED	<code>rules.ts</code> ; test
Weight and volume caps; weekly fuel quota; workshop vehicles	<code>OVER_WEIGHT</code> , <code>OVER_VOLUME</code> , <code>OVER_FUEL</code> , <code>VEHICLE_UNAVAILABLE</code>	IMPLEMENTED	<code>rules.ts</code> ; tests
Every deferral has a reason and is unavoidable or choice	<code>Deferral.reason</code> , <code>Deferral.type</code> ; Suggest plan marks deferrals	IMPLEMENTED	schema; test "suggest plan on the official peak day"
Order cutoff 16:00; times in Asia/Colombo	Cutoff and working-day logic in the API; times shown in Colombo time on every screen	IMPLEMENTED	<code>reference.ts</code> ; no automated test for the cutoff
Priority policy	Skipped last run, Fresh chilled, Fresh ambient, Tech, Style	IMPLEMENTED	<code>engine/priority.ts</code> ; test
Official peak-day scenario	Engine serves 72 of 85 orders on 32 trips and defers 13 (reproduced 2026-10-04); README states the official checker reports FEASIBILITY: PASSED	IMPLEMENTED	<code>scripts/run-task2b.ts</code> ; checker not re-run (needs Python)

17.4 Screens from the Figma frames

Original requirement	Current implementation	Status	Evidence / location
Dispatcher D1–D6	Overview, Order Queue, Planning Board, Deferrals, Live Tracking, Capacity Forecast	IMPLEMENTED	<code>roles/dispatcher</code> ; <code>docs/screens/d*.png</code>
Loader L1–L5	Choose vehicle, Load goods, Report a problem, Plan changes, Seal & release	IMPLEMENTED	<code>roles/loader</code>

Original requirement	Current implementation	Status	Evidence / location
Driver M1–M8 incl. offline	All screens; M7 is the Sync tab and M8 a sheet (see departures)	IMPLEMENTED	<code>roles/driver</code> , <code>offline/</code>
Store manager SM1–SM9 and desktop D-SM1–D-SM9	All screens; no separate store problem form	PARTIALLY IMPLEMENTED	<code>roles/store</code> ; README departures

17.5 Added after the original design

Feature	Implementation	Status	Evidence / location
Administrator role and Admin Console	System, Vehicles & drivers, Accounts, Demo mode	ADDED LATER	<code>roles/admin</code> , <code>routes/admin.ts</code>
Personal accounts, one driver per vehicle	Unique derived passwords; unique <code>Vehicle.driverId</code>	ADDED LATER	ai-disclosure; schema
Demo mode	Peak day 25 Mar 2026 with role clocks, checklist, reset	ADDED LATER	<code>apps/api/src/demo.ts</code>
Real-time refresh of all screens	SSE <code>/api/changes</code>	ADDED LATER	<code>routes/realtime.ts</code> , <code>src/live.tsx</code>
Real Asia/Colombo date and clock; current-run logic	Replaces the fixed demo day	CHANGED	<code>reference.ts</code>
Fingerprint / Face ID app lock; <code>/get-app</code> ; Android APK	WebAuthn local lock; install page; TWA build workflow	ADDED LATER	<code>biometric.ts</code> , <code>GetApp.tsx</code> , <code>android-app.yml</code>
Dispatcher depot lock; single dispatcher with switcher	Fixed-depot accounts see only their depot	ADDED LATER	<code>routes/dispatch.ts</code> (<code>dayOf</code>)
Live Tracking "Simulate morning"	Button removed from the screen; API route remains	CHANGED	git commit 3b1c40f; <code>live.ts</code>

18. Design departures

18.1 Significant departures from the original design

Each departure is taken from the README list "Departures from the Designathon design", [docs/handoff.md](#) or [docs/ai-disclosure.md](#). Where no reason is written down, that is stated.

1. Four shared logins became 188 personal accounts

Original design	Four seeded accounts (@waypoint.demo) sharing one DEMO_PASSWORD (handoff.md).
Current design	One account per administrator, dispatcher, loader, vehicle and store with a derived password; pnpm credentials lists them.
Why it changed	Reason not explicitly documented in the provided source.
Benefit	The records can name the person: Trip.loaderName , PlanAck.userName , vehicle accounts showing the assigned driver (observable in the schema).
Impact	Users need their own password; see the authentication finding in the Testing document.

2. Added Administrator role

Original design	Four roles: dispatcher, loader, driver, store.
Current design	A fifth role manages accounts, drivers on vehicles, system health and demo mode.
Why it changed	Reason not explicitly documented in the provided source.
Benefit	Driver assignment, account switch-off and password reset are possible without database access.
Impact	A fifth sign-in area and 13 admin routes.

3. One dispatcher account for both depots

Original design	Two per-depot dispatcher accounts (an intermediate state recorded in the AI disclosure).
Current design	One dispatcher@waypoint.lk with a depot switcher; accounts created with a depot stay locked to it.
Why it changed	Reason not explicitly documented in the provided source.
Benefit	One person can cover both depots.
Impact	Depot selection moved into the console.

4. Fixed demo day became real Sri Lanka time, with Demo mode on request

Original design	A fixed demo day (<code>DEMO_DATE</code> default 2026-03-25) and fixed clocks 06:42 / 02:40 / 14:18 (handoff.md).
Current design	Every screen uses today's Asia/Colombo date and clock; each day is filled with the peak-day scenario; the fixed day is available as <code>DEMO_DATE</code> or Admin → Demo mode.
Why it changed	The README says the screens run on real time "so all four roles agree".
Benefit	All roles show the same date and time; the system never runs dry of orders.
Impact	Judges can run either real time or the scripted peak day.

5. Dispatcher, loader and driver work on the "current run"

Original design	The working day was a single day.
Current design	Today until 16:00, then the next operating day.
Why it changed	AI disclosure: so that store orders reach the order queue.
Benefit	A store's order appears in the queue and can be planned the same evening.
Impact	The date on planning screens changes at 16:00.

6. Live Tracking: route column shows brand · district

Original design	Figma frame shows origin → destination.
Current design	Brand · district.
Why it changed	The booklet rule of one district per trip (README).
Benefit	Matches the data model.
Impact	Visual difference from Figma.

7. Single-district trips in loader and driver screens

Original design	Mock-ups show multi-district trips such as "Gampaha → Colombo → Kalutara".
Current design	Trips follow one brand and one district.
Why it changed	The booklet rule (README).
Benefit	Rules enforced consistently.
Impact	Mock-up examples differ from live data.

8. Demo vehicle and store

Original design	Figma uses VEH041 / VEH035.
Current design	Driver account VEH024 and store OUT026.
Why it changed	So one story runs through all four roles (README).
Benefit	A single end-to-end story.
Impact	Demo mode later places the live order on whichever vehicle the engine chooses.

9. M7 and M8 are a tab and a sheet; L5 gains a seal-number field; store problem form absent

Original design	M7 and M8 as separate screens; seal "scanned"; separate store problem form.
Current design	Sync tab plus offline strip; sheet that opens after sending; typed seal number; "Report a problem" and "Call dispatcher" open the phone dialler.
Why it changed	README lists them as departures; for L5 it notes the rationale says the seal is scanned. For the others: Reason not explicitly documented in the provided source.
Benefit	Fewer screens; simpler entry.
Impact	Minor UI differences.

10. Sidebar name and icons

Original design	Figma shows "Nimal Perera" and placeholder squares.
Current design	Signed-in dispatcher (seeded Ruwan Perera) and line icons.
Why it changed	Reason not explicitly documented in the provided source.
Benefit	Real data and icons.
Impact	Cosmetic.

11. Real-time refresh for every role

Original design	Server-Sent Events for dispatcher screens (<code>/api/events</code>).
Current design	All roles refresh through <code>/api/changes</code> .
Why it changed	AI disclosure: every screen updates without a browser refresh. Reason not explicitly documented in the provided source.
Benefit	No manual reload.
Impact	Signal-only stream; data refetched per user.

12. Simulate morning removed

Original design	README describes a Simulate morning button in Live Tracking.
Current design	Button removed (commit 3b1c40f); API route remains.
Why it changed	Commit message: Demo guide drops the tip. Reason not explicitly documented in the provided source.
Benefit	Progress comes from real driver events.
Impact	README walkthrough step 15 is out of date.

13. Deployment target and database port

Original design	DigitalOcean in docs/deploy.md ; PostgreSQL on a standard port.
Current design	AWS Lightsail per the AI disclosure; host port 15432.
Why it changed	Port: avoid clashes with local PostgreSQL on 5432/5433 (handoff.md, docker-compose.yml comments). Host: Reason not explicitly documented in the provided source.
Benefit	No local port clash.
Impact	Use <code>DB_HOST_PORT</code> to change.

19. AI disclosure

19.1 Purpose of AI usage

The repository records that AI coding assistance was used to build the project quickly: scaffolding, features, deployment scripts and documentation. `CLAUDE.md` sets rules for AI coding agents (stay in one role folder, API shapes only from `contract.ts`, planning rules only in the engine, run `pnpm test`, no raw hex colours) and requires one disclosure line after each task in `docs/ai-disclosure.md`.

19.2 AI tools used

- **Claude (Anthropic)** is the only AI tool named in `docs/ai-disclosure.md` (all 16 entries).
- Claude Code (model Sonnet 5.5) was also used on 2026-10-04 to generate this documentation package; that entry is added to the disclosure log (last row below).
- Other tools: *Not verified from the provided project files.*

19.3 Dates and contributors

The disclosure covers 2026-10-02 to 2026-10-04. Entries are attributed to **Member 1–4** (the roles in the README repository table: Member 1 engine, API and database; Member 2 dispatcher screens; Member 3 loader and driver apps; Member 4 shared UI and store manager). Personal names are not recorded in the disclosure table. `docs/handoff.md` additionally names M.M.B.V. Mudalige as a teammate who merged dispatcher UI changes and a new `Login.tsx`. No other names are stated.

19.4 What AI helped with

- Monorepo scaffold, planning engine, tests, contract, Prisma schema, seed, Docker files and README (2026-10-02).
- Dispatcher, loader, driver and store APIs and screens from the Figma designs; goods lines; plan versions; offline driver app.
- Deployment: server setup script, mobile page and Android workflow, AWS Lightsail deployment, fixes found during pre-deployment checks.
- Later features: personal accounts, administrator console, current-run logic, Colombo time, real-time refresh, Demo mode, `.dockerignore`.
- This documentation package (PDF set).

19.5 What the team implemented and verified

The "Changed by hand" column of every entry is "—": the disclosure records no hand changes. The "Checked by" column records how each result was checked (tests, end-to-end runs in Chromium against an in-memory stand-in, later on real PostgreSQL in Docker, and against the live HTTPS URL). Several early entries say they were *not yet run* on real PostgreSQL; the 2026-10-04 entries record that run.

19.6 Testing performed

See the Testing and Verification Report. In short: the disclosure records 21 engine tests passing, typecheck, official checker run, full four-role flows with contract validation on Docker and on the live URL; this document's generation re-ran the 21 tests, typecheck and web build (all passed) and the engine on the official peak day.

19.7 Human responsibility

The disclosure entries ask the team to run `docker compose up --build` and the walkthrough before merging (2026-10-02 entries), and the project rules require a disclosure line for every task. Final responsibility for the submission, its security and its accuracy rests with the team members named in the submission. *Not verified from the provided project files* as to who reviewed each AI-generated change.

19.8 AI limitations observed in the record

- Early work was verified only against an in-memory database stand-in because the AI sandbox could not install packages; later entries record real PostgreSQL runs.
- The Android workflow had never run when first written (handoff.md) and was fixed after deployment.
- Documentation drift exists between README, handoff and code (see Project Documentation); and the login handler accepts any non-empty password (see Testing report). AI-generated text and code therefore need human review.

19.9 Final verification statement

This document preserves the factual content of `docs/ai-disclosure.md` as of 2026-10-04. It does not claim that AI wrote any code beyond what that file states, and it names no contributor beyond those documented.

19.10 Disclosure log (from docs/ai-disclosure.md)

Date	Member	AI generated	Changed by hand	Checked by
2026-10-02	Member 1	Claude (Anthropic) scaffolded the monorepo, planning engine, tests, contract, Prisma schema, seed, Docker files and README from our build plan	—	13 engine tests pass; engine output on the Task 2B peak day passes the official <code>check_allocation.py</code>
2026-10-02	Member 2	Claude (Anthropic) built Part 2: trip timetable in the engine (+5 tests), dispatcher contract shapes, API routes (queue, board, suggest, check-placement, defer, confirm, publish, recovery what-ifs, live, simulate, forecast, SSE) and screens D1–D6 from the Figma file	—	18 engine tests pass; official checker still PASSED; full dispatcher flow run against an in-memory database stand-in and each screen screenshotted at 1440×900. Not yet run on real PostgreSQL/ <code>pnpm install</code> (package registry blocked in the AI's sandbox) — run <code>docker compose up --build</code> and the walkthrough before merging

Date	Member	AI generated	Changed by hand	Checked by
2026-10-02	Member 3	Claude (Anthropic) built Part 3: goods lines (+2 tests), loader API and screens L1–L5, driver route and idempotent sync API, offline driver app M1–M8 (IndexedDB outbox, service worker, manifest), plan-version snapshots and trip locking	—	20 engine tests pass; loader → seal → driver depart → deliver → offline report → reconnect flow run end-to-end in Chromium against an in-memory database stand-in, including a real offline reload served by the service worker and duplicate-safe resend. Not yet run on real PostgreSQL / <code>pnpm install</code> / a physical phone
2026-10-02	Member 4	Claude (Anthropic) built Part 4: store contract and API (today, catalogue, place/edit order before cutoff, order detail with tracking/POD/deferral, receipt confirmation, deferral accept/cancel, receipts, notifications), store screens SM1–SM9 / D-SM1–D-SM9 from the Figma file, store ordering from the goods catalogue (+1 test), dispatcher View POD, judge walkthrough, <code>docs/deploy.md</code> and <code>docs/video-script.md</code>	—	21 engine tests pass; dispatcher publish → store accepts deferral and orders → loader damage report and seal → driver delivers → store confirms with a shortage → dispatcher sees Store issue and POD, run end to end in Chromium against an in-memory database stand-in, phone and desktop screenshots. Not yet run on real PostgreSQL / <code>pnpm install</code> — run <code>docker compose down -v && docker compose up --build</code> and the walkthrough before merging
2026-10-02	Member 3	Claude (Anthropic) added the public "Get the app" page (<code>/get-app</code> : Android install/APK download, iPhone Add to Home Screen), APK and app-link serving in nginx and the service worker, <code>scripts/server-setup.sh</code> (one-command server install) and <code>docs/mobile.md</code>	—	Page checked in Chromium at phone and desktop size; script syntax-checked (<code>bash -n</code>). The APK itself is generated with PWABuilder after the site is live
2026-10-04	Member 3	Claude (Anthropic) added fingerprint / Face ID unlock to the driver app (WebAuthn app lock: <code>roles/driver/biometric.ts</code> , <code>BiometricLock.tsx</code>), the "Android app" GitHub workflow that builds the APK/AAB with Bubblewrap (<code>.github/workflows/android-app.yml</code> , <code>scripts/twa-manifest.mjs</code>) and rewrote <code>docs/mobile.md</code>	—	Fingerprint flow tested in Chromium with a virtual authenticator: turn on, lock on reopen, wrong finger rejected, unlock, turn off. Workflow YAML validated; the APK build itself runs on GitHub and has not been run yet
2026-10-04	Member 3	Claude (Anthropic) checked the whole stack before deployment and fixed <code>/sw.js</code> answering 403 in the web image (<code>apps/web/Dockerfile</code>) now makes the served files world-readable)	—	<code>pnpm install</code> , <code>pnpm test</code> (21 pass), <code>pnpm typecheck</code> clean; on real PostgreSQL in Docker, all 4 logins and every endpoint called over HTTPS with each response validated against <code>contract.ts</code> (dispatcher suggest/publish → loader claim/tick/problem/seal → driver sync incl. idempotent replay → POD → store order), on both the demo day and today

Date	Member	AI generated	Changed by hand	Checked by
2026-10-04	Member 3	Claude (Anthropic) deployed to AWS Lightsail (https://3-106-77-179.sslip.io ; images built on the Mac for x86 because the 512 MB server cannot build), fixed the Android workflow (Bubblewrap SDK layout, build-tools 36.1.0), built the APK and published it with assetlinks.json on <code>/get-app</code>	—	Full four-role flow with contract validation passed against the live HTTPS URL, then the live database was reset; APK and assetlinks served with the right content types
2026-10-04	Member 3	Claude (Anthropic) replaced the 4 shared logins with 188 personal accounts (60 vehicle/driver accounts with one unique assigned driver each, 120 store managers, 1 dispatcher + 3 loaders per depot) with unique HMAC-derived passwords and pnpm credentials ; moved every screen to today's Asia/Colombo date and real clock with automatic daily scenario build (prisma/accounts.ts , prisma/demo-day.ts); locked dispatchers to their own depot	—	21 tests, typecheck clean; on a fresh Docker database all 188 accounts sign in with their own password, old logins rejected, all roles show the same date and clock, dispatchers cannot open the other depot, no driver on two vehicles, full four-role flow passes, next-day build tested twice (idempotent)
2026-10-04	Member 3	Claude (Anthropic) added the administrator role and Admin Console (roles/admin : System, Vehicles & drivers, Accounts), admin API (routes/admin.ts), contract shapes for it, the Driver table with one driver per vehicle (unique in the database), account on/off with immediate session cut-off, password reset shown once, staff account creation, and a safe upgrade step (prisma/pre-push.sql)	—	Typecheck clean, 21 tests pass; upgrade on an existing database and start from an empty one both checked; API tests for every admin action (assign, refuse a driver on two vehicles, move with confirmation, add driver, new account, reset password, switch off incl. open session, cannot switch self off, non-admin blocked), all kept after an API restart; screens checked in headless Chrome at desktop and phone size with no console errors
2026-10-04	Member 3	Claude (Anthropic) replaced the two per-depot dispatcher accounts with one dispatcher (dispatcher@waypoint.lk) who switches between Peliyagoda and Kandy in the console; the admin can create dispatchers for both depots	—	Typecheck clean; old dispatcher logins removed on upgrade, new account has no fixed depot, overview/board follow the switcher (API and headless Chrome)
2026-10-04	Member 3	Claude (Anthropic) made the dispatcher, loaders and drivers work on the current run (today until the 16:00 cutoff, then the next run) so store orders reach the order queue, listed every order already in for the next run, and fixed the daily order build failing inside the API (data/ folder looked up from apps/api)	—	Typecheck clean, 21 tests; order cycle tested at 10:00 and 18:00 for both depots (store order listed for the next run, then in the queue, planned by auto-allocate), 85 demo orders built for the new run, four-role flow on the next run passes
2026-10-04	Member 3	Claude (Anthropic) put every on-screen time in Sri Lanka time whatever the device time zone (colomboTime / colomboHour in ui/format.ts , used by driver, loader and dispatcher screens), set the server and the api/web/caddy containers to Asia/Colombo (database stays UTC), and the setup script now sets the server time zone and internet time sync	—	Web typecheck clean; containers show +0530; four-role flow passes; live server clock checked against Colombo time

Date	Member	AI generated	Changed by hand	Checked by
2026-10-04	Member 3	Claude (Anthropic) made every screen update in real time without a browser refresh: <code>GET /api/changes</code> (SSE "something changed" signal after any successful change, at the 16:00 run switch and at midnight; no business data), <code>src/live.tsx</code> (refetch on signal, catch-up after a dropped connection or when the tab returns), the driver app refetches its route on the signal; a build id with <code>/version.json</code> shows a "New version available · Update" bar after a deploy (never reloads by itself); fixed the driver app showing the driver name from the login token instead of the vehicle's assigned driver	—	All roles received the signal ~260 ms after a store order; 5 quick changes → 1 signal; reads send none; in headless Chrome a new store order appeared on the open dispatcher queue in 630 ms with no reload, an admin switch-off showed on the open Accounts screen in 620 ms, a driver swap showed on the open driver phone in 620 ms; update bar shown after a fresh build and gone after Update; typecheck, 21 tests and the four-role flow pass
2026-10-04	Member 3	Claude (Anthropic) added Demo mode (Admin → Demo mode): start for 30 min–4 h, auto-ends or End demo now, Reset demo data; the whole system runs on Wed 25 Mar 2026 (peak day) with store 14:18 the day before, loader 02:40, dispatcher 06:42; store OUT026's dry order is left out so it is placed live; a live checklist follows the story (order → plan → deferrals and store answer → loading → both trips → receipt issue → reply); last-served dates are put back when it ends. Also: a store's deferral answer now reaches the dispatcher on the run it was deferred from	—	Typecheck clean, 21 tests; the full demo story rehearsed twice end to end through the API with real accounts (plan identical each reset: OUT026 order on VEH031 trip 2, 10 unavoidable deferrals), every step passes, End demo returns all roles to real time; Demo page checked in headless Chrome
2026-10-04	Member 3	Claude (Anthropic) merged the team's latest commits and added <code>.dockerignore</code> (no laptop <code>node_modules</code> , <code>dist</code> , <code>.git</code> , <code>.env</code> or credentials in images)	—	Typecheck and 21 tests pass; images built and the deploy checked
2026-10-04	(documentation task; member to be named)	Claude Code (Sonnet 5.5) generated the nine-PDF documentation package from the repository files	—	Re-ran 21 tests, typecheck (after <code>prisma generate</code>), web build, engine on the peak day; cross-checked schema, scripts, compose and routes against the text

20. Limitations

20.1 Limitations and known issues

Area	Status	Evidence
Monorepo, contract, schema, seed, planning engine, login API, web shell (Part 1)	IMPLEMENTED	README status checklist; 21 engine tests pass (run on 2026-10-04)
Dispatcher API and screens D1–D6 (Part 2)	IMPLEMENTED	README checklist; <code>apps/api/src/routes/dispatch.ts</code> , <code>insights.ts</code> , <code>live.ts</code>
Loader L1–L5 and offline driver app M1–M8 (Part 3)	IMPLEMENTED	README checklist; <code>routes/loader.ts</code> , <code>routes/driver.ts</code> , <code>roles/loader</code> , <code>roles/driver</code>
Store manager SM1–SM9 and API, judge walkthrough, deployment guide, video script (Part 4)	IMPLEMENTED	README checklist; <code>routes/store.ts</code> , <code>docs/deploy.md</code> , <code>docs/video-script.md</code>
Administrator console, per-person accounts, Demo mode, real-time refresh	ADDED LATER	<code>docs/ai-disclosure.md</code> entries of 2026-10-04; <code>routes/admin.ts</code>
Run on a real server	IMPLEMENTED	ai-disclosure entry "deployed to AWS Lightsail"; the URL answered HTTP 200 on 2026-10-04. The README checklist item is still unticked.
Record the video and submit	NOT VERIFIED	README checklist item is unticked; a script exists (<code>docs/video-script.md</code>), a recorded video is not among the files

Known documentation drift in the repository

- `README.md` still describes a **Simulate morning** button in Live Tracking. The latest commits remove it from the screen (commit 3b1c40f); `useSimulate` in `roles/dispatcher/api.ts` is no longer used, while `POST /api/live/simulate` remains in the API.
- `README.md` quotes both "72 of 85 orders on 32 trips" (engine on the official peak day, reproduced on 2026-10-04) and "75 of 85 orders on 34 trips" (in the app, explained by the README as a difference in the skipped-shop flag).
- `docs/handoff.md` still lists four shared logins (`@waypoint.demo`) and a shared `DEMO_PASSWORD` ; they were replaced by personal accounts on 2026-10-04.
- `docs/deploy.md` describes DigitalOcean; the AI disclosure records the live deployment on AWS Lightsail.
- The untracked folder `prisma/migrations/20261004071614_init` creates 17 tables and does not contain `Driver` or `Setting` ; the Docker setup uses `prisma db push` , not migrations.

20.2 Known limitations

- No automated API, frontend or end-to-end tests are stored in the repository.

- Sign-in accepts any non-empty password (finding F-01); a credentials CSV is tracked in git (F-04).
- Recorded video and public repository visibility are not verifiable from the files.
- The original Designathon proposal was not provided, so the requirements comparison uses the requirements recorded in the repository.

21. Future improvements

21.1 Improvements suggested by the project record

- Fix the authentication finding and rotate secrets (see Testing).
- Store the API and end-to-end test scripts in the repository and run them in CI.
- Fingerprint unlock for the store manager (listed as optional polish in `docs/handoff.md`); QR code on `/get-app`.
- Update the README walkthrough (Simulate morning, vehicle and count details) and commit `docs/data-model.md`.
- Run the Android workflow against the live domain and publish the APK (handoff).

Beyond this list, planned improvements: Not verified from the provided project files.

22. Conclusion

22.1 Conclusion

Waypoint Connect implements the full order-to-receipt flow for four roles plus an administrator, with a shared planning engine, an offline-first driver app, per-day automatic order building and a scripted demo. The planning engine reproduces the official peak-day allocation (72 of 85 orders on 32 trips, 13 deferred with reasons) and its 21 tests pass. Two issues should be fixed before the system is relied upon: permissive sign-in and a tracked credentials file. All other statements in this document are taken from the repository as of the generation date.